

“Digital Order Management with Database System and Retail Analytics”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

K. SAI SAMYUKTHA 2520030443

ANUSHKA KULKARNI 2520030501

RAKESH MEDIKONDA 2520030035

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **“Digital Order Management with Database System and Retail Analytics”** is a record of Bonafide work of **K. SAI SAMYUKTHA - 2520030443, ANUSHKA KULKARNI - 2520030501, MEDIKONDA RAKESH - 2520030035** submitted in partial fulfillment for the award of B. Tech in Computer Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K. SAI SAMYUKTHA– 2520030443

ANUSHKA KULKARNI – 2520030501

MEDIKONDA RAKESH - 2520030035

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Digital Order Management with Database System and Retail Analytics”** is a bonafide work done and submitted by **K. SAI SAMYUKTHA - 2520030443, ANUSHKA KULKARNI - 2520030501, MEDIKONDA RAKESH - 2520030035** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Yerragudipadu Subbarayudu**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. SAI SAMYUKTHA– 2520030443

ANUSHKA KULKARNI – 2520030501

MEDIKONDA RAKESH - 2520030035

Abstract

The Digital Order Management and Retail Analytics Portal is a web-based application developed to improve and simplify retail and e-commerce operations through a centralized management system. Traditional retail businesses often face challenges such as manual order processing, inventory management issues, delayed deliveries, inefficient return handling, and limited insights into customer purchasing behavior. This project aims to address these problems by integrating order management, inventory control, customer engagement, and business analytics into a single platform.

The proposed system provides secure, role-based access for administrators, staff, and customers using JWT authentication. It supports the complete order lifecycle, including product browsing, order placement, processing, shipping, delivery, cancellation, and returns. The system also includes inventory monitoring, low-stock alerts, customer profile management, notifications, and a points-based loyalty and rewards program to improve customer satisfaction.

An interactive analytics dashboard, developed using Chart.js, presents sales reports, revenue trends, order statistics, product performance, and return patterns. The Business Decision Support module analyzes sales, inventory, and return data to generate useful insights that help businesses make informed decisions.

The portal is developed using React.js for the frontend, Node.js and Express.js for the backend, and MySQL with Sequelize ORM for database management. By automating retail processes and providing data-driven insights, the system aims to reduce manual errors, improve operational efficiency, and enhance the overall shopping experience.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	22-27
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	28-30
5	Design	5.2 ER Diagram / Database schema	31
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	32-43
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	44-50
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	51-59
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	60-66
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	67-74
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	75-77
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	78-80
13	References	- Books, tutorials, APIs, documentation sites used	81

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	82-88
----	------------	---	-------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The development of digital technologies has significantly transformed the retail and e-commerce industry. As businesses increasingly depend on online platforms to sell products and manage customers, the need for organized and automated order management systems has become essential. Retail businesses deal with several interconnected activities, including product management, inventory maintenance, customer communication, order processing, delivery tracking, and return handling. When these operations are managed manually or through disconnected systems, businesses may experience delays, inaccurate records, stock availability issues, and difficulties in monitoring their overall performance.

The Digital Order Management and Retail Analytics Portal is proposed as an integrated web-based solution to address these operational challenges. The project focuses on developing a centralized platform where customers, staff, and administrators can perform their respective activities through secure, role-based access. By combining different retail functionalities into one system, the portal aims to improve coordination between customers and business operations while reducing unnecessary manual effort.

The major areas addressed by the project include:

- Order Management: Simplifies the process of placing, processing, shipping, delivering, cancelling, and returning orders.
- Inventory Management: Helps monitor product availability, track stock updates, and identify products that require restocking.
- Customer Management: Maintains customer profiles, order history, and relevant information for improved service.
- Customer Engagement: Provides order notifications and a loyalty-based points and rewards system to encourage customer interaction.
- Retail Analytics: Presents sales, revenue, product performance, and order trends through interactive dashboards.
- Business Decision Support: Uses sales, inventory, and return information to identify useful business patterns and support operational planning.

A significant aspect of this project is the integration of database management with retail analytics. A well-structured database is required to store and organize information related to users, products, orders, payments, inventory, and returns. MySQL is used as the database management system, while Sequelize ORM helps facilitate communication between the backend application and the database. This integration supports the management of related information and helps maintain consistency across different retail operations.

The project also introduces a Business Decision Support module that uses available business data to generate meaningful insights. For example, identifying fast-selling products can help businesses plan stock replenishment, while recognizing frequently returned products can highlight areas that require further attention. Similarly, monitoring revenue and order trends can provide a clearer understanding of business performance. These insights are intended to support informed business planning based on the data available within the system.

The portal is developed using React.js for frontend development, Node.js and Express.js for backend services, and MySQL with Sequelize ORM for database management. JWT authentication is used to support secure user access, while Chart.js is incorporated to display information through visual charts and dashboards.

Overall, the Digital Order Management and Retail Analytics Portal combine database systems, backend development, frontend technologies, and business analytics to create a comprehensive retail management solution. The project is intended to improve the organization of retail processes, provide useful business insights, and create a convenient and reliable experience for both customers and business administrators.

1.2 Problem Statement

The retail and e-commerce industry handles a large volume of product, customer, order, inventory, and return-related information. Managing these activities without an integrated system can result in operational inefficiencies, inaccurate records, delays in order processing, and difficulties in monitoring business performance. Businesses may face challenges in maintaining updated inventory, tracking orders, handling cancellations and returns, and understanding customer purchasing patterns. The absence of a centralized platform can also make it difficult for administrators and staff to access accurate information and make timely decisions.

The major problems identified in the existing retail management process include:

- **Manual Order Processing:** Managing orders manually can increase processing time and the possibility of errors.
- **Inventory Management Issues:** Difficulty in tracking stock levels can lead to stock shortages and inefficient inventory planning.
- **Order and Return Tracking:** Monitoring order status, cancellations, deliveries, and returns can become complicated without a proper system.
- **Limited Customer Insights:** Businesses may lack sufficient information about customer preferences, order history, and purchasing behavior.
- **Inadequate Business Analytics:** Without interactive dashboards, it can be difficult to monitor sales, revenue, product performance, and return trends.
- **Delayed Decision-Making:** The lack of combined sales, inventory, and return data can make it challenging to identify important business trends and plan operational activities.
-

To address these problems, there is a need for a centralized Digital Order Management and Retail Analytics Portal that integrates retail operations with database management and analytical features. The proposed system will provide

role-based access for administrators, staff, and customers, allowing each user to access the features relevant to their role. It will support order management, inventory monitoring, customer engagement, loyalty rewards, and return processing through a unified platform.

The system will also incorporate an interactive analytics dashboard and a Business Decision Support module to generate useful insights from retail data. These features will help identify low-stock products, fast-selling items, slow-moving products, and frequently returned products. By combining operational management with data analytics, the project aims to reduce manual work, improve information management, enhance customer experience, and support more efficient retail business operations.

1.3 Scope of the Project

The scope of the Digital Order Management and Retail Analytics Portal is to develop an integrated web-based platform that simplifies and improves the management of retail and e-commerce operations. The project focuses on bringing different business activities together into a centralized system, allowing administrators, staff, and customers to access features according to their assigned roles.

The scope of the project includes the following areas:

- **User and Role Management:** Provides secure registration, login, and role-based access using JWT authentication.
- **Product and Order Management:** Supports product browsing, order placement, order processing, shipping, delivery, cancellation, and return handling.
- **Inventory Management:** Monitors product stock levels, updates inventory records, and identifies low-stock products.
- **Customer Engagement:** Includes customer profiles, order history, notifications, and a points-based loyalty and rewards system.
- **Analytics Dashboard:** Displays sales, revenue, order trends, product performance, and return patterns using interactive charts.
- **Business Decision Support:** Analyzes sales, inventory, and return data to identify fast-selling products, slow-moving products, and frequently returned items.
- **Database Management:** Uses MySQL and Sequelize ORM to store and manage information related to users, products, orders, inventory, and returns.
- **Frontend and Backend Development:** Utilizes React.js for the frontend and Node.js with Express.js for backend services.
- **Operational Efficiency:** Aims to reduce manual work, minimize errors, and improve coordination between customers, staff, and administrators.
- **Future Scalability:** Provides a foundation for extending the system with additional retail management and analytics features as business requirements grow.

Overall, the project aims to create a reliable and user-friendly retail management platform that combines order processing, inventory control, customer engagement, and data analytics. By integrating these functionalities, the system supports efficient business operations, improves customer experience, and helps administrators make informed decisions using available business data.

1.4 Objectives of the Project

The primary objective of the Digital Order Management and Retail Analytics Portal is to develop a comprehensive and centralized web-based platform that integrates various retail and e-commerce operations into a single, efficient, and user-friendly system. The project aims to simplify the management of products, customers, orders, inventory, and returns while reducing the dependency on manual processes. It focuses on managing the complete order lifecycle, including order placement, processing, shipping, delivery, cancellation, and returns, in an organized manner. The system also aims to implement secure role-based access using JWT authentication for administrators, staff, and customers, ensuring that users can access features according to their responsibilities. By maintaining structured and accurate database records, the project seeks to improve information management, minimize operational errors, and enhance coordination between different users of the platform.

Another important objective of the project is to improve customer satisfaction and business efficiency by providing a range of customer-oriented and inventory management features. The portal aims to support customer profile management, order history, order status notifications, and a points-based loyalty and rewards system that encourages customer engagement and improves the shopping experience. It also focuses on inventory monitoring by maintaining updated stock records and identifying low-stock products, helping businesses manage product availability more effectively. Through the integration of these functionalities, the system aims to reduce the time required for routine operations, improve the accuracy of order and inventory information, and provide a convenient platform for both customers and administrators. The project also seeks to establish a better connection between customer activities and business operations by organizing relevant data within a centralized database.

The project further aims to develop an interactive Retail Analytics Dashboard and Business Decision Support module that helps businesses understand their performance through meaningful data analysis. The analytics dashboard will use Chart.js to display sales, revenue, order trends, product performance, and return patterns in the form of interactive charts and visualizations. The Decision Support module will combine sales, inventory, and return data to identify important business trends, including fast-selling products, slow-moving products, low-stock items, and frequently returned products. These insights can assist administrators in understanding operational requirements and planning inventory and business activities more effectively. The project will utilize React.js for frontend development, Node.js and Express.js for backend services, and MySQL with Sequelize ORM for database management. Overall, the objective is to create an integrated retail management solution that improves operational efficiency, supports data-driven decision-making, enhances customer engagement, and provides a strong foundation for managing modern retail and e-commerce activities.

1.5 Importance of the Application

The Digital Order Management and Retail Analytics Portal is important because it provides an integrated solution for managing retail and e-commerce activities through a centralized web-based platform. As businesses handle increasing numbers of products, customers, and orders, an efficient management system is essential for maintaining accurate records and improving operational performance. The application helps connect different retail activities, reduce manual effort, and provide users with a convenient and organized system for managing their daily operations.

The importance of the application can be explained through the following aspects:

1. **Improved Order Management:** The application simplifies the process of placing, processing, shipping, delivering, cancelling, and returning orders. This helps businesses organize order-related activities and reduce errors caused by manual processing.
 2. **Efficient Inventory Control:** The system helps administrators and staff monitor stock availability, maintain inventory records, and identify low-stock products. This supports better stock planning and reduces the possibility of inventory-related problems.
 3. **Enhanced Customer Experience:** Customers can manage their profiles, view order history, receive notifications, and access loyalty rewards. These features improve convenience, customer engagement, and transparency throughout the shopping process.
 4. **Centralized Database Management:** The application stores important information related to users, products, orders, inventory, and returns in a structured MySQL database. This helps improve data organization and allows relevant information to be accessed more efficiently.
 5. **Data-Driven Business Decisions:** The analytics dashboard provides visual information about sales, revenue, order trends, product performance, and return patterns. This helps administrators understand business performance and identify important trends.
 6. **Business Decision Support:** The Decision Support module combines sales, inventory, and return data to provide useful insights into low-stock items, fast-selling products, slow-moving products, and frequently returned products. These insights can assist businesses in planning and managing their operations.
 7. **Improved Operational Efficiency:** By integrating multiple retail functionalities into one platform, the application reduces the need for separate management processes and helps save time and effort. It supports better coordination between administrators, staff, and customers.
 8. **Secure and Role-Based Access:** JWT authentication and role-based access ensure that administrators, staff, and customers can use the features permitted for their roles, supporting controlled access to application functionalities.
- Overall, the application is important because it combines retail management, database systems, customer engagement, and analytics into a unified platform. It aims to improve the accuracy and efficiency of retail operations while providing useful insights for business planning. Through its integrated features, the application supports a more organized shopping experience for customers and more effective management for retail businesses.

1.6 Target Users / Audience

The Digital Order Management and Retail Analytics Portal is designed to support different users involved in retail and e-commerce operations. The application provides role-based access, allowing each user to access features according to their responsibilities and requirements.

1. **Administrators:** Administrators are the primary users responsible for managing the overall retail system. They can monitor business operations, manage product and inventory information, analyze sales and revenue reports, and use the Business Decision Support module to understand business trends. The analytics dashboard helps administrators monitor performance and support operational planning.

2. **Staff Members:** Staff members use the application to manage day-to-day retail activities, including order processing, inventory updates, shipping, delivery, and return handling. The system helps staff organize order-related information, track customer requests, and improve efficiency in their assigned responsibilities.

3. **Customers:** Customers are the end users who access the platform to browse products, place orders, view order history, track order status, and manage cancellations or returns. They can also manage their profiles, receive notifications, and participate in the loyalty and rewards system to improve their shopping experience.

4. **Retail Business Owners and Managers:** Business owners and managers can use the analytics and reporting features to monitor sales performance, product demand, inventory status, and return patterns. The insights generated by the system can assist them in understanding business activities and planning future operations.

5. **E-commerce Businesses:** Small and medium-sized retail businesses can benefit from the centralized platform by managing their daily operations through a unified system. The application supports the integration of order management, inventory control, customer engagement, and analytics, providing a foundation for organized and efficient retail management.

Overall, the target audience includes administrators, staff members, customers, retail business owners, and e-commerce businesses. The application aims to meet the operational and management needs of these users by providing secure access, organized information, and useful business insights through a single platform.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Digital Order Management and Retail Analytics Portal requires suitable hardware resources to support the development, implementation, testing, and execution of the web-based application. Since the project involves frontend development, backend services, database management, and retail analytics, the computer system should have sufficient processing power, memory, and storage to run multiple development tools smoothly. The hardware configuration should support the execution of React.js, Node.js, Express.js, MySQL Server, and Visual Studio Code without significant performance issues.

The processor plays an important role in executing application code, running backend services, and managing database operations. An Intel Core i3 processor or an equivalent processor is considered suitable for basic development and testing requirements. A processor with multiple cores can help improve performance when running the frontend, backend, and database simultaneously. The system should have a minimum of 8 GB RAM to support multitasking, including code editing, running the application, managing database queries, and viewing analytics dashboards. Additional RAM may be beneficial when working with larger datasets or multiple development applications.

The computer should have at least 256 GB of SSD storage to accommodate the operating system, development software, project source code, database files, and required dependencies. SSD storage helps improve system responsiveness and reduce application loading times. The system should also include a suitable display with a minimum resolution of 1366 × 768 pixels to support frontend development and testing of the portal's user interface. A standard keyboard and mouse are required for writing code, navigating development tools, and interacting with the application.

A stable internet connection is necessary for downloading software packages, installing libraries, accessing documentation, and testing web-based functionalities. The computer should be capable of running Node.js and MySQL Server alongside the frontend and backend components. Sufficient storage and reliable hardware are important for maintaining project files, database records, and testing data throughout the development process.

The following hardware specifications are considered suitable for the project:

Hardware Component Requirement

Processor	Intel Core i3 or equivalent
RAM	Minimum 8 GB
Storage	256 GB SSD or higher
Operating System	Windows 10/11 or equivalent
Display	Minimum 1366 × 768 resolution

Input Devices	Standard keyboard and mouse
Network	Stable internet connection
Development System	Computer capable of running Node.js and MySQL Server

Overall, these hardware requirements provide a suitable environment for developing and operating the Digital Order Management and Retail Analytics Portal. The configuration supports the integration of frontend and backend technologies, database management, and analytics-related functionalities. Higher hardware specifications may be required in the future if the application is expanded to handle larger datasets, increased user traffic, or more complex analytical operations.

2.2 Software Requirements

The Digital Order Management and Retail Analytics Portal requires a suitable set of software tools, frameworks, libraries, and development platforms to support the design, implementation, testing, and maintenance of the application. Since the project combines frontend development, backend services, database management, authentication, analytics, and version control, the software environment must support the integration of these different components. The following software technologies and tools are used to develop the application and ensure that its functionalities work efficiently.

1. Operating System

The project can be developed and executed on Windows 10 or Windows 11, which provides a suitable environment for installing and running the required development tools. The operating system supports applications such as Visual Studio Code, Node.js, MySQL Server, and Git. It also enables developers to manage project files, execute commands through the terminal, and test the application locally. Other compatible operating systems, such as Linux or macOS, can also be used if they support the required frameworks and software dependencies.

2. Frontend Technology – React.js

React.js is used for developing the frontend of the Digital Order Management and Retail Analytics Portal. It is a JavaScript library that helps create interactive and reusable user interface components. React.js is used to develop different pages and features, including customer registration, login, product browsing, shopping cart, order management, customer profiles, and administrative dashboards. Its component-based architecture helps organize the frontend code and makes it easier to maintain and update the application. React.js also supports dynamic rendering, allowing the user interface to respond to user actions and changes in application data.

3. Backend Technology – Node.js and Express.js

Node.js is used as the runtime environment for executing JavaScript on the server side. It enables the development of backend services that handle requests from the frontend and communicate with the database. Express.js is a backend framework used to create APIs, manage routes, and handle server-side operations. In this project, Node.js and Express.js support functionalities such as user authentication, product management, order processing, inventory updates, and return handling. The backend acts as a connection between the frontend interface and the MySQL database, ensuring that data is processed and transferred appropriately.

4. Database Management System – MySQL

MySQL Server is used as the relational database management system for storing and managing application data. The database maintains information related to customers, administrators, staff, products, orders, inventory, returns, and loyalty rewards. MySQL helps organize data into related tables and supports SQL queries for inserting, retrieving, updating, and deleting records. It is important for maintaining data consistency and supporting relationships between different entities in the retail system. The database also enables the backend to retrieve relevant information for order processing, inventory monitoring, and analytics.

5. Sequelize ORM

Sequelize ORM (Object-Relational Mapping) is used to simplify communication between the Node.js backend and the MySQL database. It allows developers to define database models using JavaScript and perform database operations without writing every SQL query manually. Sequelize helps manage relationships between tables, validate data, and perform CRUD operations. In this project, it can be used for managing user details, product information, order records, inventory updates, and customer-related data. It supports organized backend development and improves the maintainability of database-related code.

6. Authentication – JSON Web Token (JWT)

JSON Web Token (JWT) is used to support secure user authentication and role-based access within the application. It helps verify the identity of users after login and allows the backend to control access to protected routes. The system supports different user roles, such as administrators, staff, and customers, with access to relevant functionalities. JWT-based authentication helps protect sensitive operations and ensures that users cannot access features outside their permitted roles. This contributes to the security and reliability of the retail management portal.

7. Analytics and Visualization – Chart.js

Chart.js is used to create interactive charts and visualizations for the retail analytics dashboard. The library helps present business information in an understandable format, including sales performance, revenue trends, order statistics, product performance, and return patterns. Graphical representations make it easier for administrators to interpret data and monitor business activities. The analytics dashboard can support the Business Decision Support module by presenting information that helps identify low-stock products, fast-selling products, slow-moving items, and frequently returned products.

8. Development Environment – Visual Studio Code

Visual Studio Code (VS Code) is used as the primary code editor for developing the project. It provides features such as syntax highlighting, code completion, debugging support, terminal integration, and extensions for JavaScript, React.js, Node.js, and database development. Developers can use VS Code to create and manage frontend and backend files, execute commands, install dependencies, and identify errors during development. Its lightweight interface and customization options make it suitable for working on full-stack web applications.

9. Version Control and Collaboration – Git and GitHub

Git and GitHub are used for version control, project management, and collaboration among team members. Git is a distributed version control system that tracks changes made to the project files and allows developers to maintain different versions of the source code. It helps team members create branches, commit updates, merge changes, and restore earlier versions when required.

GitHub is an online platform used to host the project's source code and support collaboration between team members.

The project repository can contain frontend code, backend code, database-related files, documentation, and other necessary resources. Team members can upload their contributions, review changes, and work on separate branches before merging updates into the main project.

GitHub is particularly useful for this project because different team members may work on modules such as customer engagement, core e-commerce operations, analytics, and user access management. Using Git and GitHub helps maintain a common codebase, track individual contributions, reduce the risk of losing code, and support organized development. It also provides a record of project progress through commits and repository history.

10. Package Management – npm

npm (Node Package Manager) is used to install, manage, and maintain the libraries and dependencies required for the project. It supports the installation of React-related packages, Express.js, Sequelize, JWT libraries, and other backend or frontend dependencies. The package.json file helps maintain information about the project, including required packages and available scripts. npm simplifies dependency management and allows developers to install the necessary software packages while setting up the development environment.

11. Web Browser

A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required to run and test the web-based application. The browser allows developers and users to access the frontend interface, interact with different features, and verify the behaviour of the application. Browser developer tools can be used to inspect webpage elements, identify frontend errors, monitor network requests, and test responsive layouts. This helps ensure that the portal functions properly and provides a suitable user experience.

12. GitHub Repository and Project Documentation

GitHub also supports the documentation and organization of the project by allowing the team to maintain a README file, setup instructions, project details, and development-related information. The repository can be used to describe the technologies implemented, explain how to run the application, and provide information about the project structure. Maintaining documentation helps team members understand the system and makes it easier to continue development or troubleshoot issues in the future.

2.3 Development Tools and Frameworks

The Digital Order Management and Retail Analytics Portal is developed using various tools and frameworks that support frontend development, backend programming, database management, data visualization, and team collaboration. These technologies help in building an organized, secure, and efficient web-based retail management application. Each tool plays an important role in implementing the project functionalities and ensuring smooth communication between different components of the system.

1. React.js

React.js is a JavaScript library used for developing the frontend of the application. It helps create interactive and reusable user interface components. In this project, React.js is used to develop customer pages, product displays, shopping cart functionality, order management screens, customer profiles, and administrative dashboards. Its component-based structure makes the application easier to maintain and update.

2. Node.js

Node.js is a JavaScript runtime environment used for executing backend code. It allows the development team to build

server-side services and handle requests from the frontend. In this project, Node.js supports backend operations such as user authentication, order processing, inventory management, and communication with the database.

3. Express.js

Express.js is a backend web application framework built on Node.js. It is used to create APIs, manage server routes, and process client requests. The framework helps connect the React.js frontend with the MySQL database through backend services. Express.js is used for implementing functionalities such as customer management, product operations, order processing, and return handling.

4. MySQL Server

MySQL Server is the relational database management system used to store and manage project data. It maintains information related to users, products, orders, inventory, customer profiles, and returns. MySQL supports SQL queries and relationships between tables, helping the application retrieve and update information efficiently.

5. Sequelize ORM

Sequelize ORM is used to establish communication between the Node.js backend and MySQL database. It allows developers to define models and perform database operations using JavaScript. Sequelize helps manage relationships, create records, retrieve information, and update database entries. It improves the organization and maintainability of database-related code.

6. JSON Web Token (JWT)

JWT is used for authentication and role-based access control. It helps verify user identity after login and allows the application to protect specific routes and functionalities. In this project, JWT supports access management for administrators, staff, and customers, ensuring that users can access the features assigned to their roles.

7. Chart.js

Chart.js is a data visualization library used to create interactive charts and graphs. It is implemented in the analytics dashboard to display sales, revenue, order trends, product performance, and return patterns. The visual representation of data helps administrators understand business performance and identify important retail trends.

8. Visual Studio Code

Visual Studio Code is the primary development environment used for writing and managing the project's source code. It supports JavaScript, React.js, Node.js, and other development technologies through extensions and built-in features. Developers can use its integrated terminal, debugging tools, and code editor to create, test, and maintain the application.

9. Git

Git is a distributed version control system used to track changes in the project's source code. It allows developers to create commits, manage branches, and maintain different versions of their work. Git helps the team monitor development progress and restore previous versions of files when necessary.

10. GitHub

GitHub is a cloud-based platform used for hosting the project's source code and supporting collaboration between team members. The team can upload code, review changes, manage repositories, and combine contributions from different members. GitHub is particularly useful when team members work on separate modules, such as customer engagement, analytics, and order management. It helps maintain a shared codebase and supports organized project development.

11. npm (Node Package Manager)

npm is used to install and manage the dependencies required for frontend and backend development. It helps install

libraries, frameworks, and packages used in the project. npm also supports project scripts and dependency management through the package.json file, making it easier to set up and maintain the development environment.

Conclusion

The development tools and frameworks used in the Digital Order Management and Retail Analytics Portal provide the necessary support for creating a full-stack web application. React.js manages the frontend interface, while Node.js and Express.js handle backend services. MySQL and Sequelize ORM support database operations, and JWT provides authentication. Chart.js enables data visualization, while Visual Studio Code, Git, and GitHub support coding, version control, and team collaboration. Together, these technologies help the development team build, manage, and maintain an integrated retail management system.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end design of the Digital Order Management and Retail Analytics Portal is developed using React.js to create an interactive, user-friendly, and responsive web application. The frontend serves as the main interface through which customers, staff, and administrators interact with the system. It is designed to provide easy navigation, clear presentation of information, and convenient access to different functionalities based on the user's assigned role. The design focuses on creating an organized interface that improves the overall user experience and simplifies retail management activities.

React.js is used to develop reusable components, allowing different sections of the application to be created and maintained efficiently. The frontend includes features such as customer registration, login, product browsing, shopping cart management, order placement, order history, return management, and loyalty rewards. Customers can interact with the application to view products, place orders, track order status, and manage their profiles. Administrators and staff members can access relevant management interfaces to monitor orders, inventory, and other business operations.

The frontend communicates with the backend services through APIs, enabling the exchange of information between the user interface and the database. Axios can be used to send HTTP requests from the React.js application to the backend developed using Node.js and Express.js. This communication allows the frontend to retrieve product information, submit orders, update customer details, and display the latest data received from the server.

The front-end design also includes an interactive analytics dashboard developed using Chart.js. This dashboard presents important business information such as sales, revenue, order trends, product performance, and return patterns through charts and visual representations. The dashboard is intended to make complex business data easier to understand and help administrators monitor retail activities.

The main features of the front-end design include:

- **User-Friendly Interface:** Provides simple navigation and organized layouts for different users.
- **Role-Based Pages:** Displays features according to the access permissions of administrators, staff, and customers.
- **Product and Cart Interface:** Allows customers to browse products, add items to the cart, and place orders.
- **Order Management Interface:** Displays order details, status updates, cancellations, and return-related information.
- **Customer Engagement:** Provides customer profiles, notifications, order history, and loyalty rewards.
- **Analytics Dashboard:** Uses interactive charts to represent sales, revenue, and other retail performance metrics.
- **Responsive Design:** Aims to provide a convenient interface across different screen sizes and devices.

Overall, the front-end design plays an important role in connecting users with the functionalities of the Digital Order Management and Retail Analytics Portal. By using React.js, Axios, and Chart.js, the application aims to provide an interactive and organized interface that supports retail operations, customer engagement, and business analytics.

3.2 Back-End Design

The backend design of the Digital Order Management and Retail Analytics Portal is developed using Node.js and Express.js to manage server-side operations, process user requests, and establish communication between the frontend and the database. The backend acts as the core processing layer of the application, responsible for handling business logic, managing data, providing APIs, and ensuring that different functionalities operate efficiently. It connects the React.js frontend with the MySQL database using Sequelize ORM, enabling the system to store, retrieve, and update information related to customers, products, orders, inventory, and returns.

Node.js is used as the runtime environment for executing JavaScript code on the server side, while Express.js provides the framework for developing backend services and RESTful APIs. These APIs allow the frontend to communicate with the backend and perform different operations based on user requests. The backend is designed to process requests, validate information, apply business rules, and return appropriate responses to the frontend. This separation between frontend and backend helps organize the application and makes the system easier to maintain and expand.

The backend includes authentication and authorization mechanisms using JSON Web Token (JWT) to provide secure access to the application. When a user registers or logs in, the backend verifies the credentials and manages authentication through tokens. Role-based authorization ensures that administrators, staff, and customers can access only the functionalities permitted for their respective roles. This helps protect application data and restrict unauthorized access to important operations.

The backend manages the complete order lifecycle, including order placement, processing, shipping, delivery, cancellation, and returns. It also handles product information, inventory updates, customer profiles, order history, notifications, and loyalty rewards. Through Sequelize ORM, the backend interacts with MySQL database tables and performs operations such as creating new records, retrieving information, updating existing data, and deleting records when required. The database structure supports relationships between different entities, such as customers, products, orders, and inventory.

The backend also supports the Analytics Dashboard and Business Decision Support module by collecting and processing relevant retail data. It can retrieve sales information, revenue records, order trends, product performance, and return details from the database. This information is provided to the frontend, where Chart.js is used to display visual analytics. The Decision Support module combines sales, inventory, and return data to generate insights related to low-stock products, fast-selling items, slow-moving products, and frequently returned products.

The main components of the backend design include:

- **API Development:** Creates backend endpoints for communication between the frontend and database.
- **Authentication and Authorization:** Implements JWT-based authentication and role-based access control.
- **Order Processing:** Handles order placement, status updates, cancellations, delivery, and returns.
- **Inventory Management:** Processes stock updates, product availability, and low-stock monitoring.
- **Database Connectivity:** Uses Sequelize ORM to communicate with the MySQL database.
- **Customer Management:** Handles customer profiles, order history, notifications, and loyalty points.
- **Analytics Support:** Retrieves and processes business data for dashboards and decision-making.
- **Error Handling and Validation:** Validates requests and manages errors to improve system reliability.

3.3 Database Design

The database design of the Digital Order Management and Retail Analytics Portal focuses on organizing, storing, and managing the information required for retail and e-commerce operations. The database acts as an important component of the application by maintaining records related to customers, administrators, staff, products, orders, inventory, returns, and loyalty rewards. A well-structured database helps ensure that information is stored systematically and can be accessed whenever required by the frontend and backend services. The project uses MySQL Server as the relational database management system and Sequelize ORM to establish communication between the backend and the database.

The database is designed to support different functionalities of the portal, including user authentication, product management, order processing, inventory monitoring, customer engagement, and retail analytics. Different tables are used to store relevant information, and relationships between tables help maintain consistency and reduce unnecessary duplication of data. For example, customer details can be connected with their orders, while order information can be associated with the products purchased by the customer. This structure allows the application to retrieve relevant information efficiently and maintain relationships between different retail activities.

The database design supports the following major areas:

1. **User Management:** The database stores information about administrators, staff, and customers, including their identification details, login credentials, and assigned roles. This information supports user authentication and role-based access control through JWT.
2. **Product Management:** Product-related details, such as product names, categories, prices, descriptions, and availability, are maintained in the database. These records allow customers to browse products and help administrators manage product information.
3. **Order Management:** The database stores order details, including customer information, order items, order dates, total amounts, and order status. It supports the management of orders throughout the lifecycle, from placement and processing to delivery, cancellation, and returns.

4. **Inventory Management:** Inventory records are maintained to monitor product stock levels and availability. The system can use this information to update stock when orders are placed and identify products that require restocking through low-stock alerts.

5. **Customer Engagement and Loyalty:** Customer profiles, order history, notifications, and loyalty points are managed through relevant database records. These features support customer engagement and the rewards system.

6. **Return Management:** The database maintains information related to product returns, including order references, return requests, statuses, and relevant details. This helps organize the return process and supports tracking of return-related activities.

7. **Analytics and Decision Support:** Sales, inventory, and return data stored in the database can be retrieved and processed to support analytics dashboards and business decision-making. The system can use this information to identify sales trends, product performance, low-stock items, and frequently returned products.

Sequelize ORM is used to define models, establish relationships, and perform database operations through the Node.js backend. It supports common operations such as creating, reading, updating, and deleting records. The use of Sequelize helps simplify database interaction and improves the maintainability of the application's backend code.

The database design aims to maintain accurate and consistent retail information, reduce data redundancy, support efficient data retrieval, and provide a reliable foundation for the application. It also supports secure access to relevant information and allows the system to generate useful data for analytics and business decision support. The database can be expanded in the future to accommodate additional features and growing business requirements.

Overall, the database design provides the foundation for the Digital Order Management and Retail Analytics Portal by organizing important retail information into related tables. Through MySQL Server and Sequelize ORM, the system supports efficient data storage, retrieval, and management. The database enables the frontend and backend components to work together effectively while supporting order processing, inventory control, customer engagement, and retail analytics.

3.4 Version Control

Version control is an important part of software development that helps developers manage, track, and maintain changes made to the project's source code. It allows multiple team members to work on the same project while maintaining an organized record of all modifications. In the Digital Order Management and Retail Analytics Portal, Git is used as the version control system to manage project files and support collaboration among team members.

Git maintains a history of changes made to the frontend, backend, database configuration, and other project files. Whenever a developer adds a new feature, modifies existing code, or fixes an error, the changes can be recorded using Git. This makes it easier to understand the development progress and identify when a particular change was introduced. During the development of the project, team members can work on different modules, such as customer engagement,

loyalty management, analytics, and admin dashboard features. Version control helps combine these contributions while reducing the risk of accidentally overwriting another team member's work. It also provides a structured way to manage different versions of the application.

Tools and Features Used

- Git: Used to track changes and manage different versions of the source code.
- Repositories: Used to store and organize project files in one location.
- Commits: Used to save changes with a description of the work completed.
- Branches: Allow developers to work on different features separately.
- Merging: Combines changes from different branches into the main project.
- Change History: Helps developers review previous modifications and restore earlier versions when required.

Advantages of Version Control

1. Improved Collaboration: Enables team members to work together efficiently on different project modules.
2. Change Tracking: Maintains a record of modifications made to the source code.
3. Error Recovery: Allows developers to return to a previous working version if a problem occurs.
4. Data Protection: Reduces the possibility of losing important project code.
5. Efficient Development: Organizes the development process and simplifies project maintenance.
6. Better Project Management: Helps monitor the progress of different features and updates.

Overall, Git provides a reliable and organized method for managing the source code of the Digital Order Management and Retail Analytics Portal. By using version control, the development team can collaborate effectively, maintain code consistency, track project progress, and reduce the risks associated with frequent code changes.

3.5 APIs or External Services

APIs (Application Programming Interfaces) and external services play an important role in the development of the Digital Order Management and Retail Analytics Portal. They enable communication between the frontend, backend, and database, allowing different components of the application to exchange information efficiently. APIs act as a connection between the React.js frontend and the Node.js and Express.js backend. When a user performs an action, such as logging in, viewing products, placing an order, or checking order details, the frontend sends a request to the backend through the appropriate API.

The backend receives these requests, processes the required operations, and communicates with the MySQL database using Sequelize ORM. After completing the operation, the backend sends a response to the frontend, where the result is displayed to the user. This communication process helps maintain a proper separation between the user interface, application logic, and database management.

The project uses APIs for various functions, including user registration, authentication, product management, order processing, inventory management, customer engagement, loyalty points, and analytics. The User Management API supports registration and login functionality, while the Product Management API handles product details, product availability, and updates. The Order Management API supports order placement, retrieval of order information, and order status management. Customer engagement APIs help manage customer profiles and loyalty-related features. Analytics APIs retrieve information such as sales, revenue, order trends, product performance, and returns for displaying on the admin dashboard.

Several technologies and external services are used to support API communication and application functionality. Axios is used in the React.js frontend to send HTTP requests to the backend and receive responses. JSON Web Token (JWT) supports user authentication and helps control access to protected application features. Sequelize ORM provides communication between the backend and MySQL database by using models and database operations. Chart.js is used to represent analytical information through graphs and charts, helping administrators understand business performance and make informed decisions.

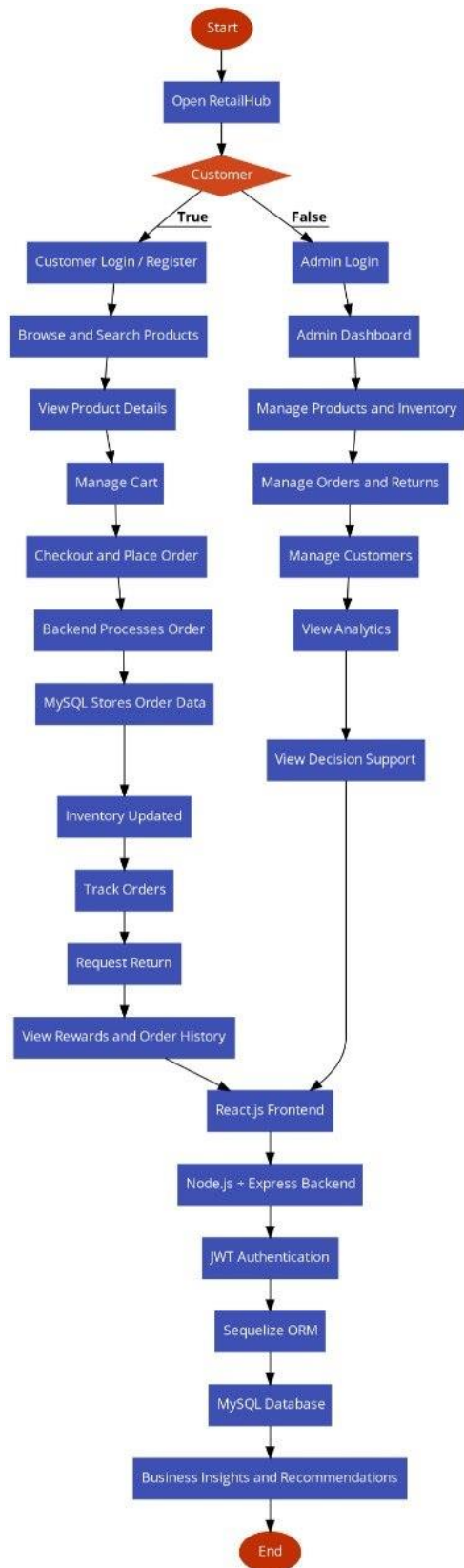
The API communication process begins when a user interacts with the frontend application. The frontend sends the request to the Express.js server using Axios. The backend then validates the request, performs the required business logic, and interacts with the MySQL database through Sequelize. Once the operation is completed, the backend returns the result to the frontend. The frontend processes the response and displays the required information to the user. This process ensures smooth data exchange and supports the efficient functioning of the application.

The use of APIs and external services provides several benefits, including modular development, improved communication, code reusability, scalability, and easier maintenance. It allows different modules of the system to work together while reducing unnecessary manual operations. APIs also support centralized data processing and help maintain consistency between the frontend and backend. Authentication mechanisms provide controlled access to system features, while analytics services improve the presentation of business information.

Overall, APIs and external services are essential for developing a reliable and efficient Digital Order Management and Retail Analytics Portal. They connect the major components of the application and support important functionalities such as customer management, product handling, order processing, loyalty management, and business analytics. Their integration improves system flexibility, performance, maintainability, and user experience.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

Frontend Layer

The **Frontend Layer** represents the user interface of the Digital Order Management and Retail Analytics Portal. It is developed using **React.js** and is responsible for providing an interactive and user-friendly environment through which customers and administrators can access the system. The frontend acts as the first point of interaction between the user and the application. When the application is opened, users are directed to the appropriate functionality based on whether they are customers or administrators.

For customers, the frontend provides features such as **registration and login, browsing and searching products, viewing product details, managing the shopping cart, checking out, placing orders, tracking orders, requesting returns, and viewing rewards and order history**. These features allow customers to complete the complete shopping process through a single interface. The frontend also displays the information received from the backend in an organized manner so that users can easily understand and interact with the system.

For administrators, the frontend provides access to the **Admin Dashboard**, where they can manage products and inventory, manage orders and returns, manage customer information, and view analytics. The dashboard helps administrators monitor different aspects of the retail system and access business-related information. The frontend sends user requests to the backend and displays the results returned by the server.

Overall, the Frontend Layer focuses on **user interaction, navigation, data presentation, and communication with the backend**. React.js helps organize the interface into reusable components, making the application easier to maintain and update. It provides a common interface through which both customer operations and administrative operations can be performed efficiently.

Backend Layer

The **Backend Layer** acts as the processing and application logic layer of the system. It is developed using **Node.js and Express.js** and works between the React.js frontend and the MySQL database. The backend receives requests from the frontend, processes them according to the required operation, communicates with the database, and sends the appropriate response back to the frontend.

The backend handles major operations related to the customer and administrator modules. When a customer places an order, the request is received by the backend and the required order information is processed. The backend communicates with the database to store the order details, after which the inventory information can be updated. Similarly, operations such as product management, order management, returns, customer management, and analytics are processed through the backend.

The system also uses **JWT authentication** for managing secure access to the application. It helps the system

authenticate users and control access to features that require authorization. This is particularly important for administrator functionality because administrative operations should only be accessible to authorized users.

The backend also uses **Sequelize ORM** to communicate with the MySQL database. Instead of directly handling every database operation from the frontend, the backend manages the communication and performs the required operations through Sequelize. This provides a structured connection between the application logic and stored data.

The Backend Layer therefore acts as the **central processing unit of the application**. It manages requests, performs application operations, handles authentication, communicates with the database, and provides processed information to the frontend. This separation makes the system more organized and allows frontend and backend functionalities to be managed independently.

Database Layer

The **Database Layer** is responsible for storing, organizing, and managing the data required by the Digital Order Management and Retail Analytics Portal. The system uses **MySQL** as the database management system. It provides persistent storage for information generated during customer and administrator activities.

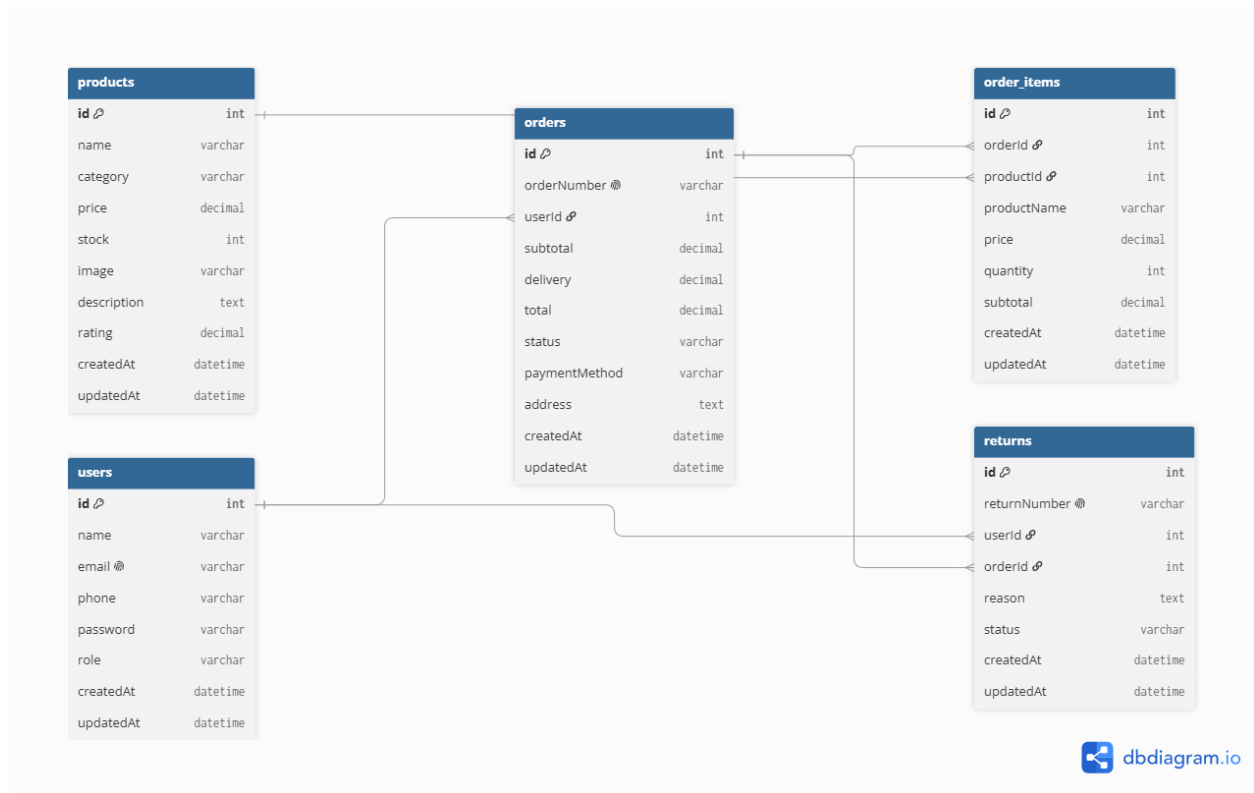
The database stores important information related to **customers, products, orders, inventory, returns, rewards, and order history**. When a customer registers or places an order, the corresponding information is stored in the database. Similarly, product and inventory information can be maintained and updated based on the operations performed through the system.

For example, when a customer places an order, the order details are processed by the backend and stored in MySQL. The inventory information can then be updated according to the order. The stored order and customer information can also be used when the customer wants to track an order, request a return, or view previous orders and rewards.

The database also supports the **analytics and decision-support features** of the system. Information stored during different transactions can be processed to generate business insights and recommendations. Administrators can use this information through the dashboard to understand aspects such as sales, orders, inventory, product performance, and returns.

The Backend Layer communicates with the MySQL database using **Sequelize ORM**. This allows the application to perform operations such as creating, retrieving, updating, and managing records in a structured way. The database layer ensures that information is stored systematically and remains available whenever it is required by the application. Overall, the Database Layer provides the **data storage and management foundation** of the entire system. It supports customer operations, administrative operations, order processing, inventory management, analytics, and decision support. Together with the React.js frontend and Node.js/Express.js backend, the MySQL database forms a three-layer architecture that allows the Digital Order Management and Retail Analytics Portal to function as an integrated system.

CHAPTER -5 DESIGN



This ER diagram represents the database design of the Digital Order Management and Retail Analytics Portal.

- Users & Customers: Store user login and customer information.
- Products & Categories: Store product details, prices, categories, and stock.
- Orders & Order Items: Manage customer orders and the products included in each order.
- Payments: Stores payment details related to orders.
- Inventory: Tracks product stock and low-stock alerts.
- Returns: Manages product return and refund information.
- Rewards & Reviews: Manage customer loyalty points and product feedback.
- Admin: Supports administrative management of the system.
- Analytics: Stores information used for sales, order, and business analysis.

The relationships between these tables show how customer orders, products, payments, inventory, returns, and analytics are connected. In conclusion, the diagram provides the database structure required to support the main operations of the project.

CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

The implementation of the **Digital Order Management with Database System and Retail Analytics** was structured into several interrelated modules that together form a centralized digital platform for managing retail and e-commerce activities. Each module was designed with a clearly defined functional responsibility while maintaining interaction with the other modules through the backend services and centralized database. This modular approach makes the system easier to develop, maintain, test, and extend because changes made to one functional area can be managed without unnecessarily affecting unrelated components of the application.

The main objective of the implementation was to create a unified environment in which different retail activities could be handled through a single system. Traditional retail management may involve separate activities for maintaining product information, processing orders, monitoring inventory, maintaining customer information, managing returns, and analysing business performance. Managing these activities independently can result in duplicated work, inconsistent information, and difficulty in understanding overall business performance. The proposed system brings these activities together so that information generated during normal retail operations can also be used for reporting and decision-making.

The implementation consists of five major functional modules: **User Access and Role Management, Core E-Commerce Operations, Customer Engagement and Loyalty, Analytics and Admin Dashboard, and Decision Support System**. These modules are not implemented as completely isolated components. Instead, they exchange information through the application backend and database. For example, when a customer places an order, the order information becomes part of the customer's history, affects product and inventory information, contributes to sales data, and can later be used by the analytics and decision-support modules.

The application follows a layered web-based implementation in which the frontend is responsible for user interaction, the backend is responsible for application logic and API processing, and the database is responsible for persistent data storage. The frontend is developed using **React.js**, providing a component-based interface for different types of users. The backend is developed using **Node.js and Express.js**, which handle API requests, validation, authentication, business operations, and communication with the database. **Sequelize ORM** is used to provide structured interaction between the backend application and **MySQL Server**. **JWT** is used for authentication and controlled access, while **Chart.js** is used to present analytical information through interactive visualizations.

The modular implementation also provides a foundation for future improvements. New retail functionality can be introduced by extending the relevant module and connecting it with the existing backend and database structure. This reduces the need to redesign the complete application whenever additional functionality is required. The separation of responsibilities also improves maintainability because developers can identify the appropriate module when modifying or troubleshooting a particular feature.

The implementation gives particular importance to the relationship between transactional operations and analytical operations. Product and order management generate the primary business data, while customer and loyalty features

provide information about user engagement. This information is subsequently used by the analytics dashboard and decision-support functionality. As a result, the application is not limited to performing transactions but also provides a foundation for understanding business performance and supporting operational decisions.

6.1.1 User Access and Role Management

The **User Access and Role Management** module forms the security foundation of the Digital Order Management and Retail Analytics system. The purpose of this module is to provide controlled access to the application and ensure that different users are provided with functionality according to their assigned roles. The system supports three major categories of users: **administrators, staff members, and customers**. Each category interacts with the system according to its operational requirements.

The implementation begins with user registration. A new user provides the required account information through the registration interface. The information entered through the frontend is sent to the backend using an API request. The backend receives the request and performs the required validation before storing the user information in the database. This separation ensures that user data is not directly inserted into the database from the frontend and that application-level processing can be performed before the information is stored.

The login functionality is implemented to verify the identity of registered users. When a user enters their login credentials, the frontend sends the information to the backend. The backend checks the submitted credentials against the stored user information. If the credentials are valid, the authentication process is completed and a JSON Web Token is generated for the authenticated user.

JWT provides a secure mechanism for identifying authenticated users during subsequent requests. After successful login, the token is used when the user accesses protected resources. The backend verifies the token before processing requests that require authentication. This prevents users who are not authenticated from directly accessing protected functionality.

The role-management functionality is implemented along with authentication because simply identifying a user is not enough for a system that contains multiple categories of users. After identifying the authenticated user, the application determines the role associated with that account. The role is then used to control access to different features.

Administrators have access to management-oriented functionality required for controlling products, inventory, orders, analytics, and other administrative activities. Staff members receive access to the operational functionality necessary for handling retail activities. Customers receive access to customer-oriented features such as viewing available products, placing orders, managing their profiles, checking order history, receiving order updates, and using loyalty-related functionality.

Role-based access is important because it protects the application from unauthorized operations. For example, a customer should not be able to modify inventory information or access administrative business analytics. Similarly, operations that can affect the complete retail system should only be available to authorized users.

The implementation of authentication and authorization is therefore integrated into the backend rather than relying only on frontend restrictions. Although the frontend can hide or display features based on the user's role, the backend remains responsible for verifying permissions before allowing protected operations. This provides an additional security layer because users cannot gain unauthorized access simply by manipulating the frontend interface.

The User Access and Role Management module therefore provides a controlled entry point into the complete application. It ensures that user identity is verified, access permissions are respected, and different categories of users can interact with the system according to their responsibilities.

6.1.2 Core E-Commerce Operations

The **Core E-Commerce Operations** module represents the central operational component of the system. This module manages the primary retail activities associated with products, orders, inventory, and returns. Since these operations generate the majority of transactional information used by the application, this module also provides important data for the analytics and decision-support components.

Product management is implemented to maintain structured information about products available through the retail platform. Authorized users can manage product-related information through the application interface. Product records are stored in the MySQL database so that they can be retrieved whenever customers or authorized staff need to interact with them.

Maintaining product information in a centralized database makes it easier to keep the system consistent. Instead of maintaining separate product records for different parts of the application, the same database information can be used by product displays, order processing, inventory monitoring, and analytical operations. This reduces duplication and provides a common source of product information.

Order management is one of the most important parts of the module. The system is designed to manage the complete order lifecycle, beginning with order placement and continuing through processing, shipping, delivery, cancellation, and returns. Each stage provides information about the current condition of an order and allows both customers and authorized users to understand its progress.

When a customer places an order, the frontend collects the required order information and sends it to the backend. The backend validates the request and processes the order according to the application rules. The corresponding order information is then stored in the MySQL database. This creates a persistent record that can later be accessed through customer order history, administrative interfaces, and analytical functions.

Order status management is important because an order changes over time. A newly placed order may initially be under processing. After processing, it may proceed to shipping and later delivery. If an order is cancelled or returned, the corresponding status and related information are updated. Maintaining these changes provides a complete record of the order lifecycle.

Inventory management is closely connected with order processing. Product availability depends on the amount of stock maintained by the system. When an order is successfully processed, the relevant inventory information can be updated so that the available quantity remains consistent with actual transactions.

The system also supports low-stock monitoring. When the available quantity of a product becomes low, the information can be identified by the application and presented for administrative attention. This feature is useful because inventory shortages can affect order fulfillment and customer satisfaction.

Return and refund processing forms another important part of the core operations. Customers may need to return products under appropriate conditions, and the system maintains information about these return activities. The return information remains associated with the corresponding order so that the organization can maintain a record of returned products and understand return patterns over time.

The return information also has importance beyond the individual transaction. Since the analytics and decision-support modules use operational data, information about returned products can be analysed to identify products that are frequently returned. This demonstrates how the core operational module contributes directly to the analytical capabilities of the system.

Overall, the Core E-Commerce Operations module provides the transactional foundation of the application. It ensures that products, orders, inventory, cancellations, and returns are handled in a centralized and structured manner while generating reliable information that can be used by the other modules.

6.1.3 Customer Engagement and Loyalty

The **Customer Engagement and Loyalty** module is implemented to improve the relationship between customers and the retail platform beyond the basic process of purchasing products. A digital retail system needs to provide customers with convenient access to their personal information, previous orders, order updates, and loyalty benefits. This module brings these customer-oriented functions together in one area.

Customer profile management allows customer information to be maintained in an organized manner. The profile is associated with the customer's account and provides a way to manage information required by the application. Maintaining customer information in a centralized database allows other modules to associate transactions and activities with the correct customer.

Customer history is another important part of the module. When customers place orders, the corresponding information becomes part of their purchase history. This allows customers to review their previous activities and provides a record of their interaction with the retail platform.

Order status notifications are implemented to improve communication between the system and customers. Customers do not need to repeatedly search for their order information because important changes in order status can be

communicated through the notification functionality. Notifications can provide information related to the progress of an order and help customers remain aware of the current status.

The notification mechanism also supports the broader objective of improving customer satisfaction. Clear communication reduces uncertainty after an order has been placed. Customers can understand whether an order is being processed, shipped, delivered, cancelled, or associated with a return activity.

The loyalty component introduces a **points-based loyalty and rewards system**. Customer purchasing activity can contribute to loyalty points according to the rules implemented by the application. The accumulated points can then be associated with the rewards mechanism provided by the platform.

The loyalty functionality is intended to encourage continued engagement with the system. Customers receive an additional benefit from their purchasing activity, while the business gains a structured way of maintaining loyalty-related information.

The module also provides useful information for understanding customer behaviour. Customer profiles, purchase history, and loyalty information can be considered together to understand how customers interact with the platform. This information can support the broader analytics capabilities of the system.

The Customer Engagement and Loyalty module therefore combines profile management, purchase history, notifications, and loyalty functionality into a unified customer experience. It helps the system move beyond basic order processing and provides features designed to maintain customer engagement and improve the overall retail experience.

6.1.4 Analytics and Admin Dashboard

The **Analytics and Admin Dashboard** module is implemented to provide a centralized view of the business information generated by the retail system. Retail applications generate large quantities of data through product sales, customer orders, inventory changes, and returns. Presenting this information only as individual records makes it difficult to understand broader business trends. The analytics dashboard addresses this issue by presenting important information in a summarized and visual form.

The dashboard provides information related to **sales, revenue, order trends, top-selling products, product performance, and return patterns**. These areas provide administrators with a broader understanding of how the business is performing.

Sales information allows administrators to examine the performance of products and overall transactions. Revenue information provides another perspective by showing the financial contribution generated by the orders stored in the system. Order trends can be used to understand changes in order activity over the available data.

Product performance analysis provides information about which products are performing strongly and which products

may require further attention. Top-selling products can be identified based on sales activity, allowing administrators to understand which products are contributing significantly to customer demand.

Return patterns provide another important analytical perspective. Returns are not simply operational events; when analysed across multiple transactions, they can reveal patterns that may require business attention. Frequently returned products can be examined further by administrators.

The backend is responsible for retrieving the relevant information from the database and preparing the data required by the dashboard. The frontend then receives the processed information through the appropriate API and presents it through visual components.

Chart.js is used to create interactive charts and visualizations within the dashboard. Visual representation makes numerical information easier to understand because users can identify comparisons, trends, and changes without examining every individual database record.

The dashboard also provides a centralized administrative interface. Instead of moving through multiple screens to examine different types of information, administrators can obtain an overview of business performance from the analytics section. This improves the efficiency of monitoring and allows important conditions to be identified more easily.

The analytics dashboard is also closely connected with the Decision Support System. The dashboard focuses on presenting business information in a visual format, while the decision-support component uses related information to identify patterns and generate useful recommendations.

The Analytics and Admin Dashboard therefore acts as the reporting and visualization layer of the application. It converts transactional information stored in the database into understandable business information that can support monitoring and decision-making.

6.1.5 Decision Support System

The **Decision Support System** is implemented to transform the information generated by normal retail operations into useful business insights. The system combines information from sales, inventory, and returns to identify important patterns that may require administrative attention.

A retail organization generates different types of information through its daily operations. Sales records show customer demand, inventory records show product availability, and return records show products that customers return. When these data sources are examined together, they can provide more useful information than when they are considered separately.

One of the important purposes of the Decision Support System is to identify **low-stock products**. Inventory information can be examined to determine which products have reached a low level. Such information helps

administrators identify products that may require replenishment.

The system also identifies **fast-selling products**. Products with high sales activity can be recognized as products experiencing strong demand. This information can be useful for inventory planning because products with high demand may require closer monitoring.

Another important category is **slow-moving products**. These products have relatively lower sales activity and may remain in inventory for longer periods. Identifying such products helps administrators understand which products require additional attention.

Return information provides another source of decision-support information. Products that are returned frequently can be identified from return records. A high return frequency may indicate that the product needs further investigation or review.

The Decision Support System therefore combines different business dimensions rather than depending on a single type of data. This provides a broader perspective of business activity. For example, a product with high sales but low inventory represents a different business condition from a product with low sales and high inventory.

The system is designed to present these conditions as useful business insights. The purpose is to assist administrators in recognizing important situations more quickly instead of manually examining large numbers of individual records.

The Decision Support System strengthens the analytical capabilities of the project by connecting operational data with business decision-making. It demonstrates how a database-driven retail application can be extended beyond transaction processing to support informed management decisions.

6.2 Front-End Logic

The frontend of the Digital Order Management and Retail Analytics Portal provides the main interaction layer between users and the system. **React.js** is used to develop the frontend because the application contains multiple interactive sections that need to communicate with backend services and update their content according to user actions.

The frontend is responsible for displaying the appropriate interfaces for authentication, product management, order operations, customer information, loyalty functionality, notifications, analytics, and administrative activities. Different sections of the interface are presented according to the functionality available to the authenticated user.

The component-based structure of React.js allows the interface to be divided into reusable and manageable components. Instead of implementing the entire application as one large interface, individual sections can be developed and maintained separately. This makes the application easier to modify and provides better organization of frontend functionality.

User interaction is an important part of frontend logic. When a user enters information into a form, selects a product, places an order, updates profile information, or requests analytical information, the corresponding React component processes the interaction and communicates with the backend when necessary.

The frontend communicates with backend services through API requests. When information is required from the server, the appropriate request is sent to the backend and the returned response is used to update the interface. This allows the application to display current information without requiring direct database access from the browser.

The frontend also handles the presentation of different states of the application. For example, the interface may display different information when a user is logged in compared with when the user is not authenticated. Similarly, administrative interfaces can provide different options from customer interfaces.

Form handling and validation are also important aspects of the frontend implementation. User inputs need to be collected in a structured manner before being sent to the backend. Appropriate validation can help identify incomplete or invalid information at the interface level and reduce unnecessary requests.

The analytics interface uses **Chart.js** to display business information visually. Data received from the backend can be represented using appropriate charts so that administrators can understand sales, revenue, product performance, order trends, and return information more easily.

The frontend also provides feedback to users after important operations. For example, after an order is successfully processed, the interface can display the updated order information. Similarly, when profile or product information is modified, the updated information can be reflected in the relevant interface.

Another important characteristic of the frontend implementation is its connection with role-based access. The interface can present functionality according to the authenticated user's role. However, frontend controls are not treated as the only security mechanism. The backend also verifies authentication and authorization before processing protected operations.

Overall, the frontend logic is designed to provide a clear and interactive interface while keeping the complexity of backend processing hidden from the user. React.js handles the presentation and interaction layer, while the backend remains responsible for application processing and database communication.

6.3 API Integration

The **API Integration** layer provides communication between the React.js frontend and the Node.js and Express.js backend. It acts as the main communication mechanism through which users' actions are converted into backend operations and backend results are returned to the frontend.

The application uses backend APIs for major operations such as user registration, login, product management, order processing, inventory management, return processing, customer profile management, notifications, loyalty functionality, analytics, and decision-support information.

When a user performs an operation through the frontend, the application generates an HTTP request to the appropriate backend endpoint. Express.js receives the request and routes it to the relevant backend functionality. The backend then validates the request and performs the required business operation.

The backend does not allow the frontend to communicate directly with MySQL. Instead, the backend acts as an intermediary between the user interface and database. This separation provides better control over the data and allows validation, authentication, authorization, and business rules to be applied before database operations are executed.

Node.js provides the runtime environment for the backend application, while **Express.js** provides the framework for defining and handling HTTP routes and API requests. The backend is organized around the functional requirements of the application so that different API operations can be associated with their respective modules.

When a request requires information from the database, the backend uses **Sequelize ORM** to perform the required database operation. Sequelize provides an abstraction layer between the Node.js application and MySQL Server. This allows the backend to work with structured application models rather than writing every database interaction directly at the application level.

For example, when a customer requests their order history, the frontend sends a request to the relevant backend API. The backend identifies the authenticated customer, retrieves the required order information through Sequelize, and obtains the corresponding records from MySQL. The result is then returned to the frontend, where it is displayed to the customer.

A similar process occurs during order creation. The customer submits the order through the frontend, the request is received by the Express.js backend, the information is validated, and the required records are created or updated in the database through Sequelize. The backend then returns an appropriate response to the frontend.

API integration is also important for analytics. When the administrator opens the dashboard, the frontend requests analytical information from the backend. The backend retrieves and processes the relevant business information from the database and returns the result. The frontend then uses the returned data to create visualizations using Chart.js.

The use of APIs creates a clear separation between the user interface and business logic. The frontend focuses on displaying information and collecting user actions, while the backend controls how those actions are processed. The API layer therefore serves as the communication backbone of the application. It allows the different frontend interfaces to access the required backend functionality in a structured manner and ensures that database operations remain controlled by the server-side application.

6.4 Authentication and Authorization

The **Authentication and Authorization** implementation provides security for the application by controlling who can access the system and what each authenticated user is allowed to perform. Since the application contains customer information, product information, order records, inventory data, and administrative analytics, controlling access to these resources is an important part of the implementation.

Authentication is the process of verifying the identity of a user. The process begins when a registered user enters their

login credentials through the frontend. These credentials are sent to the backend through the authentication API, where they are verified against the stored account information.

After successful verification, the backend generates a **JSON Web Token (JWT)**. The token provides an authenticated identity that can be used for subsequent protected API requests. This allows the backend to recognize the user making a request without requiring the user to provide their complete login information for every operation.

Protected API requests include the authentication information required by the backend. Before processing such requests, the server verifies the JWT. If the token is valid, the request can continue to the relevant application logic. If the token is missing, invalid, or no longer acceptable, access to the protected operation is denied.

Authorization is performed after authentication. While authentication establishes the identity of the user, authorization determines whether that user has permission to access the requested functionality.

The project uses role-based access control with administrator, staff, and customer roles. Each role has different responsibilities within the application. Administrators require access to management and analytical functions, staff members require operational functions, and customers require customer-facing functions.

For example, a customer may be allowed to view products and manage their own orders, but should not be allowed to modify system-wide inventory information. An administrator, on the other hand, requires access to broader management and analytics functionality.

Authorization checks are implemented at the backend level so that protected resources cannot be accessed simply by modifying frontend elements. This is important because frontend restrictions alone cannot provide reliable security. The backend must independently verify whether the authenticated user is authorized to perform the requested operation.

Authentication and authorization also help protect the database because users are not provided with direct access to database records. Requests must pass through the application backend, where access permissions can be evaluated before the required database operation is executed.

The security implementation therefore provides a controlled environment for the different categories of users supported by the application. By combining JWT-based authentication with role-based authorization, the system provides secure and organized access to its various modules.

6.5 Database Implementation

The database implementation provides the centralized storage layer for the Digital Order Management and Retail Analytics system. **MySQL Server** is used as the relational database management system. The database stores information generated by the different modules and provides a common source of information for both operational processing and analytical activities.

The database maintains information related to users, roles, products, orders, order-related records, inventory, customers, returns, loyalty information, notifications, and information required for analytics. These entities are logically related because retail operations depend on the interaction between customers, products, orders, inventory, and returns.

For example, customer information is associated with customer orders, while order information is associated with the products purchased. Product transactions affect inventory information, and returned products create additional information that can be used for return analysis.

The relational structure of MySQL is useful for maintaining these relationships in an organized manner. It allows information to be stored in structured tables and retrieved when required by different application modules.

Sequelize ORM is used to connect the Node.js backend with MySQL. It provides models and database operations that allow the backend to create, retrieve, update, and delete records according to the requirements of the application.

When a new record is created through the application, the frontend sends the information to the backend. The backend validates the request and uses Sequelize to perform the required database operation. Similarly, when information is requested, Sequelize retrieves the appropriate records and provides the result to the backend.

The centralized database also allows different modules to work with consistent information. An order created by the order-management module can later be used by the customer-history module, analytics dashboard, and decision-support system. This prevents each module from maintaining separate copies of the same business information.

The database therefore plays an important role in integrating the complete system. It provides persistent storage for operational information while also serving as the foundation for reporting and business analysis.

6.6 Overall System Implementation

The overall implementation integrates the frontend, backend, database, authentication mechanism, transactional modules, analytics functionality, and decision-support functionality into a single digital retail management platform. The different components are designed to work together while maintaining clear separation of responsibilities.

The implementation begins at the user interface, where users interact with the application according to their roles. Authentication establishes the user's identity, and role-based authorization controls the functionality available to that user.

Once authenticated, customers can interact with products and place orders. The order-management functionality processes the transaction and maintains the corresponding records. Inventory information is updated according to the operational activity, while customer history and loyalty information are maintained as part of the customer engagement module.

As retail operations continue, the system accumulates transactional information in the MySQL database. This information represents the operational history of the platform and provides the data required for further analysis.

The analytics dashboard uses this stored information to provide administrators with a summarized view of sales, revenue, order trends, product performance, top-selling products, and return patterns. Chart.js is used to present this information through interactive visualizations, making the information easier to understand.

The Decision Support System extends this analytical capability by examining sales, inventory, and return information together. It helps identify low-stock products, fast-selling products, slow-moving products, frequently returned

products, and other relevant business trends.

The integration of these components allows the system to perform both operational and analytical functions within the same platform. The application does not simply record transactions; it also uses the information generated by those transactions to provide a clearer understanding of business performance.

The modular implementation improves maintainability because each major area of functionality has a defined purpose. The user-management functionality handles access control, the core e-commerce module handles transactions, the customer module handles engagement, the analytics module handles visualization, and the decision-support module handles business insights.

The implementation also provides a foundation for future enhancement. Additional features can be integrated into the existing architecture by extending the appropriate frontend components, backend APIs, database structures, or analytical functionality. The separation between presentation, application logic, and data storage makes such expansion more manageable.

Overall, the implementation successfully brings together **digital order management, inventory monitoring, customer engagement, loyalty management, analytics, and decision support** within one centralized system. The combination of React.js, Node.js, Express.js, Sequelize ORM, MySQL Server, JWT authentication, and Chart.js provides the technical foundation required to implement the proposed retail management platform.

CHAPTER 7- FEATURES

CHAPTER 7 – FEATURES

7.1 List of Main Features

The Digital Order Management and Retail Analytics Portal has been conceived as a comprehensive web-based platform designed to integrate the major activities involved in modern retail and e-commerce operations within a single centralized environment. The system brings together order processing, inventory management, customer engagement, loyalty management, analytics, notifications, and business decision support through a unified interface. Instead of treating each operation as an independent activity, the platform connects these functions through a common database and backend service layer, allowing information generated during one operation to support other activities within the system. The overall objective is to reduce manual effort, improve operational accuracy, provide better visibility into business activities, and support both customers and administrative users through an organized digital environment.

Foremost among the components is the User Access and Role Management Feature, which provides controlled access to the portal according to the responsibilities of different categories of users. The system supports administrators, staff members, and customers through role-based access mechanisms. Users can register and authenticate securely before accessing the functions available to their assigned roles. Authentication is implemented using JSON Web Tokens, while authorization mechanisms ensure that users can access only the operations permitted for their respective roles. This feature establishes the security foundation of the application while also providing a personalized experience for different types of users.

The Core E-Commerce Operations Feature forms the primary functional component of the platform and manages the complete order lifecycle. Customers can interact with products, place orders, and monitor the progress of their purchases. Orders can move through different operational stages including placement, processing, shipping, delivery, cancellation, and returns. The system maintains the status of each order and ensures that the associated information is updated throughout the lifecycle. This centralized management reduces dependence on manual tracking and provides users with clearer visibility into their transactions.

Closely associated with order processing is the Inventory Management Feature, which enables the system to maintain information regarding available products and stock quantities. Inventory information is updated as orders are processed, allowing the system to provide a current representation of product availability. The feature also supports low-stock monitoring so that products approaching critical stock levels can be identified. Such monitoring assists staff and administrators in taking timely action to reduce the possibility of stock shortages and improve the overall efficiency of retail operations.

The Customer Engagement and Loyalty Feature provides functionality intended to strengthen the relationship between the customer and the retail platform. Customer profiles can be maintained within the system along with relevant order information and purchasing history. The platform also incorporates a points-based loyalty mechanism through which

customers can accumulate points based on their interactions and purchases. These points can contribute to rewards and encourage continued engagement with the platform. By combining customer information, order history, notifications, and loyalty activities, the system provides a more personalized customer experience.

The Notification Feature supports communication between the system and its users by providing relevant updates associated with orders and other important activities. Notifications can be generated when significant changes occur in the order lifecycle or when actions require user attention. This feature improves transparency by ensuring that customers and operational users are informed about important events rather than depending entirely on manual communication. The notification mechanism therefore contributes to improved responsiveness and customer satisfaction.

The Analytics and Admin Dashboard Feature provides administrators with a centralized view of important business information. The dashboard uses interactive visualizations created with Chart.js to represent sales, revenue, order trends, product performance, and return patterns. Instead of requiring administrators to interpret large quantities of raw records manually, the dashboard presents summarized information through charts and other visual representations. These analytical views help administrators understand the current state of the business and identify important patterns in retail activity.

The Business Decision Support Feature extends the analytical capabilities of the platform by combining information from sales, inventory, and return activities. The system can identify useful business conditions such as low-stock products, fast-selling products, slow-moving products, and products that are frequently returned. Based on these observations, the decision support component provides useful insights and recommendations that can assist administrators in making better operational decisions. This transforms the platform from a basic order management system into a system capable of supporting data-driven retail management.

The Return Management Feature provides a structured mechanism for handling products that customers wish to return. Return information is associated with the corresponding order and product records, allowing the organization to maintain visibility into return activities. The collected return information can also be used by the analytics component to identify return patterns and products with comparatively high return activity. Consequently, return processing is not treated only as a post-purchase operation but also contributes to the broader analytical capabilities of the platform.

Underlying these visible capabilities is the Database Management Feature, which provides centralized storage and organization of the information generated throughout the application. MySQL is used as the relational database system, while Sequelize ORM provides structured interaction between the Node.js backend and the database. User information, products, orders, inventory records, customer details, loyalty information, notifications, and analytical data are maintained through appropriate database structures and relationships. This centralized data organization ensures that different modules can work with consistent and related information.

Finally, the Integration Feature ensures that the individual components of the portal operate together as a unified system. The React.js frontend communicates with the Node.js and Express.js backend through APIs, while the backend communicates with MySQL through Sequelize ORM. Authentication and authorization mechanisms operate across the application to protect access to resources. The integration of operational data with analytics and decision

support enables information generated during routine retail activities to become useful for management and planning.

Collectively, these features represent the central objective of the Digital Order Management and Retail Analytics Portal: to create a secure, centralized, and intelligent environment for managing retail operations. Each component performs a specific responsibility while simultaneously contributing information to other components of the system. Through the combination of order management, inventory monitoring, customer engagement, loyalty services, analytics, and decision support, the platform aims to reduce operational complexity, improve accuracy, enhance customer satisfaction, and support informed business decisions.

7.3 How Each Feature Works

The operation of the Digital Order Management and Retail Analytics Portal is based on the coordinated interaction between the user interface, backend services, authentication mechanisms, database layer, and analytical components. Each feature has a specific purpose from the user's perspective while also performing defined operations within the technical architecture. The following sections describe the behaviour of the major features from both the user perspective and the technical perspective governing their execution.

1. User Access and Role Management Feature

User Perspective:

When a user accesses the portal, the system provides appropriate options for registration and login. During registration, the required user information is entered and submitted to the system. After successful registration, the user can log in using the registered credentials. Once authentication is completed, the system identifies the user's role and presents the appropriate interface and functionality. Customers receive access to customer-oriented operations, while staff and administrators receive additional operational or management capabilities according to their assigned permissions.

The role-based interface prevents unnecessary functionality from being displayed to users who do not require it. For example, administrative analytics and management functions are intended for authorized administrative users rather than ordinary customers. This creates a more organized user experience and reduces the possibility of unauthorized operations.

Technical Description:

The authentication mechanism is implemented using JSON Web Tokens. When valid login credentials are submitted, the backend verifies the credentials and generates an authentication token. The frontend uses the token when communicating with protected backend APIs. Express.js middleware validates the token and identifies the authenticated user before allowing access to protected resources. Role-based authorization is applied after authentication so that the backend can determine whether the user has permission to perform the requested operation.

2. Core E-Commerce Operations Feature

User Perspective:

Customers can use the portal to interact with available products and place orders through the digital ordering interface. After an order is submitted, the customer can monitor its progress through the order status provided by the system. The order may progress through stages such as processing, shipping, and delivery. If required, cancellation and return operations can also be initiated according to the functionality provided by the system.

The customer therefore receives a continuous view of the order lifecycle rather than depending on separate manual

communication for every stage. Staff and administrators can use their respective interfaces to manage orders and update their operational status.

Technical Description:

The order management functionality is implemented through backend APIs developed using Node.js and Express.js. When a customer places an order, the frontend sends the required order information to the backend. The backend validates the request and stores the relevant information in MySQL through Sequelize ORM. Order records are associated with the appropriate customer and product information. As staff or administrators update the order status, the corresponding database records are modified. These updated records are then retrieved by the frontend and displayed to the user.

3. Inventory Management Feature

User Perspective:

The inventory component allows authorized users to monitor product availability and stock quantities. Administrators and staff can review current stock information and identify products that are approaching low-stock levels. This provides a clear view of the products that may require replenishment or further attention.

Customers indirectly benefit from this functionality because product availability can be maintained more accurately as orders are processed. Effective inventory monitoring reduces the possibility of incorrect availability information and supports smoother order fulfilment.

Technical Description:

Inventory information is maintained within the MySQL database and accessed by the backend through Sequelize ORM. When products are involved in order transactions, the relevant stock information is updated according to the implemented business logic. The backend can evaluate stock quantities against defined low-stock conditions. Products satisfying these conditions can be identified for administrative attention. Because inventory information is maintained centrally, the same data can also be used by the analytics and decision support modules.

4. Customer Engagement and Loyalty Feature

User Perspective:

Customers can maintain their profiles and view information associated with their activities on the platform. Their order history provides a record of previous purchases, while the loyalty mechanism allows customers to accumulate points through eligible activities. The points-based reward system provides an additional incentive for continued interaction with the platform.

The customer can therefore use the portal not only for placing orders but also for managing their relationship with the retail service. Relevant notifications and account information further contribute to a personalized and convenient experience.

Technical Description:

Customer information is stored in the relational database and associated with relevant order and loyalty records. Backend APIs allow authorized users to retrieve and update permitted profile information. Loyalty points are maintained as part of the customer engagement functionality and can be updated when qualifying activities are completed. The frontend retrieves the required information through API requests and presents the customer's profile, order-related information, and loyalty details through React.js components.

5. Notification Feature

User Perspective:

The notification functionality keeps users informed about important activities within the platform. Customers can receive relevant updates related to their orders and other system events. Notifications reduce the need for users to repeatedly check the system for changes and provide timely information about activities requiring their attention.

For administrators and staff, notifications can also provide awareness of operational events such as important order or inventory-related conditions. This improves communication between the system and its users.

Technical Description:

Notifications are generated by the backend when specific events occur within the application. Relevant information is stored and associated with the corresponding user or operational record. The frontend retrieves notification information through backend APIs and displays it within the appropriate user interface. Because the notification mechanism is integrated with the main application data, messages can be associated with actual order and system events rather than being maintained as unrelated information.

6. Analytics and Admin Dashboard Feature

User Perspective:

The analytics dashboard is primarily intended for administrators who require an overview of retail operations. The dashboard presents important information such as sales performance, revenue, order trends, product performance, and return patterns. Instead of examining individual records one by one, administrators can use graphical representations to quickly understand changes and trends.

Interactive charts make it easier to compare different business indicators and identify areas that require attention. The dashboard therefore acts as a centralized management view through which administrators can observe the overall performance of the retail system.

Technical Description:

The analytics functionality obtains relevant information from the backend and database and transforms it into data suitable for graphical presentation. Chart.js is used to generate interactive visualizations within the React.js frontend. The Node.js and Express.js backend provides APIs that retrieve and summarize the required operational information. Data such as order records, sales values, revenue information, product activity, and return records can be processed to produce meaningful dashboard indicators. Administrative authorization ensures that these management-oriented analytics are accessible only to permitted users.

7. Business Decision Support Feature

User Perspective:

The Business Decision Support feature assists administrators in interpreting the information available within the retail system. Rather than simply displaying numerical data, the system highlights conditions that may require management attention. For example, administrators can identify products with low inventory, products that are selling quickly, products with comparatively low sales activity, and products that experience frequent returns.

These insights can assist administrators in making decisions related to stock management, product planning, and operational improvement. The feature therefore converts existing business data into information that is more useful for decision-making.

Technical Description:

The decision support functionality combines information obtained from sales, inventory, and return records. Backend processing evaluates these datasets to identify significant patterns and conditions. Products can be categorized according to their sales activity, inventory status, or return frequency. The resulting insights are made available to the administrative interface through APIs. Since the decision support mechanism works with the same centralized database used by the operational modules, its recommendations are based on information generated through actual system activities.

8. Return Management Feature

User Perspective:

When a customer needs to return a purchased product, the return process is associated with the relevant order. The system maintains information about the return so that both the customer and authorized staff can track the status of the request. This provides a structured alternative to handling returns through separate manual records.

Return information also contributes to the customer's order history and provides the organization with visibility into post-purchase activities. This makes the return process more transparent and manageable.

Technical Description:

Return information is stored in the database and associated with the corresponding order and product records. Backend APIs process return-related requests and apply the relevant business rules before updating the database. The stored return information can subsequently be retrieved for customer-facing status information as well as administrative analysis. Return records are also utilized by the analytics and decision support components to identify products or categories with comparatively high return activity.

9. Database Management Feature

User Perspective:

From the user's perspective, database operations occur largely in the background. When a user registers, places an order, updates permitted information, or performs another supported operation, the corresponding information is stored and retrieved automatically. Users therefore experience the system as a centralized application in which their information remains available across different sections.

The database provides continuity between different operations. For example, a customer's order can be associated with their profile, the purchased products can be related to inventory records, and return information can later contribute to analytics.

Technical Description:

MySQL Server is used as the relational database management system, while Sequelize ORM provides an abstraction layer between the Node.js backend and the database. Structured tables and relationships are used to maintain users, products, orders, inventory, customer information, notifications, loyalty information, and return-related records. Sequelize models represent the database entities and provide methods for creating, retrieving, updating, and deleting records. This architecture allows the backend to manage database operations without requiring the React frontend to directly access the database.

10. Integration and System Interoperability Feature

User Perspective:

To the user, the integration functionality appears as a continuous experience across different sections of the portal. Information entered or generated in one part of the system can be reflected in other relevant sections. For example, an

order placed by a customer becomes part of the order history, affects relevant operational records, contributes to business analytics, and can generate notifications. In a similar manner, inventory and return information can later contribute to decision support.

This interconnected behaviour allows the portal to function as a unified retail management environment rather than as a collection of unrelated pages.

Technical Description:

The frontend is developed using React.js and communicates with the backend through API requests. Node.js and Express.js provide the server-side application and REST-based services, while Sequelize ORM manages communication with MySQL. JWT authentication is applied to protected requests, and role-based authorization controls access to different operations. Operational records stored in MySQL are subsequently available to analytics and decision support services through backend processing. Chart.js provides the visualization layer for analytical information on the administrative dashboard. Through this integration, the system maintains a continuous connection between transactional operations, database management, customer engagement, analytics, and business decision support.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

The process of verifying the reliability, functional accuracy, and communication between the frontend and backend of the Digital Order Management and Retail Analytics Portal was carried out extensively using Postman, an industry-standard API testing environment. This testing phase was important for ensuring that the React.js frontend communicated correctly with the Node.js and Express.js backend services and that the data exchanged between the application and MySQL database remained accurate and consistent. The testing methodology was designed to simulate real-world retail operations, validate API behaviour under different conditions, and verify both successful and exceptional responses.

Postman served as an intermediary testing environment between the user-facing React.js application and the backend application logic. By sending HTTP requests directly to the REST API endpoints, the development team was able to test backend services independently of the graphical user interface. This approach made it easier to identify problems in individual APIs and verify request and response behaviour before integrating the services with the frontend. Environment variables were used to maintain configurable values such as the backend base URL, authentication tokens, and endpoint parameters, making the testing process repeatable during development.

A comprehensive collection of test requests was prepared in Postman to represent the major functionalities of the Digital Order Management and Retail Analytics Portal. These included user registration, user login, authentication, customer profile management, product operations, order placement, order processing, order status updates, inventory operations, notifications, loyalty points, returns, analytics data retrieval, and business decision support services. The requests were categorised according to HTTP methods such as GET for retrieving information, POST for creating records, PUT or PATCH for updating records, and DELETE for removing records. This classification provided a systematic approach for validating CRUD operations throughout the application.

For frontend validation, the API responses obtained from Postman were compared with the data expected by the React.js interface. This helped confirm that frontend components could correctly consume backend responses and display appropriate information to users. For example, when product information was returned from the backend, the frontend was expected to display product names, prices, stock information, and related details correctly. Similarly, order-related responses were checked to ensure that order status, customer information, product details, and total amounts were displayed properly. Error responses were also considered to ensure that the frontend could provide meaningful messages when an operation failed.

Backend testing involved detailed verification of the server-side business logic. Each important API endpoint was tested with both valid and invalid input data. Registration APIs were tested using valid customer information as well as incomplete or invalid data. Login APIs were tested with correct and incorrect credentials to confirm that authentication was properly enforced. Order APIs were tested with valid products and quantities as well as invalid product identifiers, unavailable stock, and missing information. These tests helped verify that the backend returned suitable HTTP status codes and meaningful response messages.

Authentication and authorisation testing was an important part of Postman testing because the system uses JWT-based authentication and role-based access. Valid JWT tokens were used to access protected endpoints, while requests without tokens or with invalid tokens were tested to ensure that unauthorised access was rejected. Administrator and staff-specific operations were also tested to confirm that only users with appropriate roles could perform sensitive operations such as inventory updates, order management, analytics access, and administrative functions. This ensured that the security rules implemented in the backend were functioning correctly.

Boundary and negative testing were also applied to important APIs. For example, order placement was tested with zero or invalid quantities, unavailable products, and incomplete customer information. Inventory-related APIs were tested with invalid stock values and product identifiers. Return functionality was tested with invalid order references and inappropriate return requests. These tests helped determine whether the backend could handle unexpected inputs without causing system failures or corrupting database records.

Postman scripts were used where appropriate to validate response codes and important response values. The testing process checked whether successful operations returned expected HTTP status codes such as 200 OK or 201 Created and whether invalid requests returned suitable error responses such as 400 Bad Request, 401 Unauthorized, 403 Forbidden, or 404 Not Found. Response data was also examined to verify that important fields such as user information, order identifiers, product details, inventory quantities, order status, and analytics values were returned in the expected structure.

Database consistency was also considered during API testing. After operations such as user registration, product creation, order placement, inventory modification, and return processing, the corresponding records were checked in the MySQL database. Sequelize ORM was responsible for communicating between the Node.js application and MySQL database. Postman testing therefore helped verify not only the API response but also whether the requested operation resulted in the correct database changes.

Testing of the analytics and business decision support modules was performed by sending requests that retrieved sales, revenue, order, product, inventory, and return-related information. The responses were checked to ensure that the React.js dashboard could receive and represent the required information through Chart.js visualisations. This helped verify the connection between transactional data stored in the database and the analytical information presented to administrators.

Security testing through Postman was also performed on protected endpoints. Requests without valid authentication credentials were used to verify that sensitive APIs could not be accessed by unauthorised users. Malformed request data and invalid parameters were also tested to ensure that backend validation and error-handling mechanisms were functioning correctly. Role-based restrictions were verified by attempting administrator-level operations using accounts with lower privileges.

The Postman collections provided a structured and repeatable method for testing the backend services throughout development. Whenever changes were made to an API or database operation, previously prepared requests could be executed again to verify that existing functionality continued to work correctly. This reduced the possibility of

introducing new errors while modifying existing features.

From a technical perspective, Postman testing helped the development team identify communication problems between the React.js frontend and Node.js backend, incorrect API responses, authentication issues, validation errors, and database-related inconsistencies. The testing process therefore acted as an important quality assurance stage before considering the complete system ready for demonstration and deployment.

To conclude, Postman testing played an important role in establishing the functional reliability of the Digital Order Management and Retail Analytics Portal. By systematically validating authentication, product management, inventory, order processing, customer services, returns, analytics, and decision-support APIs, the team ensured that data exchange between the frontend, backend, and database remained accurate, secure, and efficient. The structured testing process contributed to reducing runtime errors and improving the overall reliability of the application.

8.2 Integration Testing

Integration testing constituted an important phase in validating the overall coherence and interoperability of the Digital Order Management and Retail Analytics Portal. While individual modules could be tested separately, integration testing focused on confirming that the modules worked together correctly as a unified system. The major areas considered during integration testing included user authentication, product management, inventory management, order processing, customer engagement, loyalty points, notifications, returns, analytics, and business decision support.

The testing environment was established to represent the complete application structure consisting of the React.js frontend, Node.js and Express.js backend, Sequelize ORM, and MySQL database. Representative test data was used to simulate different retail situations, including registered customers, staff users, administrators, available products, low-stock products, placed orders, completed orders, cancelled orders, and returned orders. This allowed the system to be tested under conditions similar to actual retail operations.

A modular incremental integration strategy was adopted for the application. Initially, the communication between the React.js frontend and Node.js backend was verified. Once the basic request-response mechanism was confirmed, authentication services were integrated with user-related operations. Product and inventory services were then connected with order processing. After this, customer engagement, loyalty, notifications, returns, analytics, and decision-support features were integrated. This gradual approach made it easier to identify the source of errors when a problem occurred.

Authentication integration was one of the first areas tested. When a user logged into the system, the backend generated a JWT authentication token. The token was then used by the frontend when making requests to protected APIs. Integration testing verified that the token was correctly transferred between the frontend and backend and that protected resources were accessible only after successful authentication. Different user roles were also tested to verify that administrators, staff members, and customers received access according to their assigned permissions.

Product and inventory integration was tested to ensure that product information and stock quantities were correctly connected to order operations. When a product was available in the inventory, the customer should be able to place an order successfully. After an order was created, the corresponding inventory quantity was expected to be updated

correctly. Similarly, when the available stock reached a low level, the inventory monitoring functionality was expected to identify the condition and support the generation of a low-stock alert.

Order processing represented another major integration area. The complete order lifecycle was tested from order placement to processing, shipping, delivery, cancellation, and return. The tests verified that a change in order status was correctly stored in the database and reflected in the frontend. This ensured that customers and staff could view consistent order information throughout the order lifecycle.

The integration between customer profiles and order management was also validated. When a customer placed an order, the order was associated with the correct customer account. Customer information could therefore be used to display order history and support customer engagement features. This helped ensure that data belonging to one customer was not incorrectly associated with another user.

The loyalty and rewards functionality was integrated with order processing to verify that eligible purchases could contribute to customer points. Integration testing checked whether points were updated correctly after successful transactions and whether the corresponding information was reflected in the customer interface. This confirmed that the customer engagement module could work together with the core order management functionality.

Notification integration was tested for events that required user awareness. For example, notifications could be generated for important order updates, inventory-related conditions, or other supported system events. The testing verified that the event generated by one module was correctly processed and displayed through the notification functionality without producing duplicate or inconsistent records.

Return management integration was tested by connecting completed or eligible orders with the return process. When a valid return was initiated, the system was expected to maintain the relationship between the original order and the return information. The integration tests verified that the return status and related records were correctly stored and that the resulting information could be accessed by the appropriate users.

Analytics integration represented a significant part of the system because the analytics dashboard depended on transactional information stored in the database. Sales, revenue, order trends, product performance, inventory information, and return patterns were retrieved from the backend and presented through Chart.js in the React.js interface. Integration testing verified that changes in transactional data were correctly reflected in the corresponding analytical results.

The Business Decision Support module was tested together with the analytics and database components. The module combines relevant sales, inventory, and return information to provide useful insights for business decision-making. Integration testing verified that the module could receive the required data from the backend and generate meaningful recommendations or decision-support information without breaking the existing analytics functionality.

Database integration was continuously validated throughout the testing process. Operations performed through the frontend were checked against the corresponding MySQL records. Sequelize ORM was tested as the communication layer between the Node.js backend and the relational database. This helped ensure that records were created, updated, retrieved, and deleted correctly and that relationships between users, products, orders, inventory, returns, and other

entities remained consistent.

Error handling across integrated modules was also tested. Invalid requests, unavailable products, incorrect authentication credentials, and invalid order information were introduced to determine whether errors were handled without affecting unrelated parts of the system. The frontend was expected to display suitable error messages while the backend maintained database consistency.

Integration testing also evaluated the overall request-response flow between the application layers. When a user performed an action through the React.js interface, the request was sent to the Express.js backend, processed using the appropriate application logic and Sequelize operations, and then stored or retrieved from MySQL. The resulting response was returned to the frontend and displayed to the user. Testing this complete flow helped confirm that all layers communicated correctly.

Performance and responsiveness were considered during integration testing by executing repeated requests against important application services. The objective was to verify that the integration of frontend, backend, and database operations did not introduce unnecessary delays or failures. Particular attention was given to order operations and analytics requests because these features involve multiple database interactions.

Security integration was also verified by testing role-based access across the application. Administrator-specific operations were accessed using administrator credentials, while customer accounts were used to confirm that restricted operations could not be performed. JWT authentication and backend authorisation rules were therefore validated as part of the complete integrated workflow.

The results of integration testing helped identify issues related to data flow, authentication, database updates, API responses, and frontend rendering. Defects identified during testing were corrected and the affected integration scenarios were executed again to confirm that the changes did not negatively affect other modules.

In its concluding phase, integration testing confirmed that the major components of the Digital Order Management and Retail Analytics Portal could operate together as a unified application. The coordinated interaction between React.js, Node.js, Express.js, Sequelize ORM, and MySQL demonstrated that the system could support complete retail workflows from user access and product selection to order processing, inventory updates, customer engagement, analytics, returns, and business decision support.

8.3 Tools Used for Testing

The testing phase of the Digital Order Management and Retail Analytics Portal was supported by a set of development and testing tools that helped verify API functionality, frontend behaviour, database consistency, application performance, and version control. Each tool was used according to the requirements of the project and the technology stack. Together, these tools provided a structured testing environment for identifying defects and validating the reliability of the complete application.

1. Postman

Postman was used as the primary tool for API testing and validation. It provided an interface for creating, sending, and analysing HTTP requests and responses generated by the Node.js and Express.js backend. APIs related to

authentication, users, products, inventory, orders, returns, notifications, analytics, and decision support could be tested independently of the React.js interface.

Postman also helped verify HTTP status codes, JSON response structures, authentication headers, request parameters, and validation messages. Collections could be maintained for different application modules, making it easier to repeat important test scenarios during development.

2. MySQL

MySQL was used for database validation during testing. After performing operations through the application or APIs, the corresponding records could be inspected in the database to verify that the expected information was correctly stored. User accounts, product details, inventory quantities, orders, customer information, loyalty data, notifications, and returns could therefore be cross-checked for consistency.

MySQL validation was particularly useful during integration testing because it confirmed that successful API operations resulted in the correct database changes.

3. Sequelize ORM

Sequelize ORM was used as the database interaction layer between the Node.js backend and MySQL. During testing, Sequelize operations were validated to ensure that models, relationships, queries, and database updates worked correctly. This helped verify that application-level operations were translated into appropriate database transactions.

4. Browser Developer Tools

Browser Developer Tools were used to inspect the behaviour of the React.js frontend. The Network tab helped identify API requests and responses, while the Console helped detect frontend errors and warnings. These tools were useful for verifying whether the frontend was correctly communicating with the backend and whether returned data was being displayed properly.

5. Visual Studio Code

Visual Studio Code was used as the primary development and debugging environment. Source code for the React.js frontend, Node.js backend, Express.js APIs, and Sequelize models could be inspected and modified within the same environment. Debugging tools and terminal integration supported the execution and verification of application services during testing.

6. Git and GitHub

Git and GitHub were used for version control and collaborative development. Different changes made during development could be tracked through commits and branches. This was useful for testing because modifications could be reviewed and previous stable versions could be identified when debugging newly introduced problems.

7. Chart.js

Chart.js was used in the analytics dashboard to display visual representations of sales, revenue, order trends, product performance, and return patterns. Testing verified that the charts received correct data from the backend and represented the analytical information appropriately on the frontend.

8. React.js Development Environment

The React.js development environment was used to test user interactions and frontend workflows. Forms, navigation, dashboard components, order-related screens, customer features, and analytics components were tested through the browser to verify that the user interface responded correctly to backend operations.

9. Node.js and Express.js

Node.js and Express.js formed the backend execution environment and were therefore important during backend testing. Server logs, API requests, validation errors, authentication failures, and application responses were monitored

while executing test scenarios. This helped identify problems in server-side business logic and API routing.

10. JWT Authentication Testing

JWT authentication mechanisms were tested through API requests to verify secure access to protected resources. Valid, invalid, missing, and expired authentication tokens were considered during testing to ensure that unauthorised users could not access restricted services.

8.4 Test Cases and Results

The testing phase of the Digital Order Management and Retail Analytics Portal was carried out systematically to verify that the major functional and non-functional aspects of the application operated according to the defined requirements. Test cases were prepared for authentication, user roles, product management, inventory, order processing, customer engagement, loyalty points, notifications, returns, analytics, and business decision support. Each test case considered the required input, execution conditions, expected output, and observed result.

The user registration functionality was tested by entering valid customer information and verifying that a new account was successfully created. Invalid and incomplete information was also tested to confirm that the backend rejected unsuitable requests. The login functionality was tested using valid credentials and incorrect credentials. Successful login generated the required authentication information, while invalid credentials were rejected.

Role-based access was tested by using different user roles. Administrator and staff accounts were used to access management functions, while customer accounts were used to access customer-specific operations. The tests confirmed that restricted functionality was protected using JWT authentication and role-based authorisation.

Product management test cases verified that products could be created, retrieved, updated, and removed according to the supported application operations. The returned product information was checked for correctness and consistency. Invalid product identifiers and incomplete product data were also tested to ensure appropriate error handling.

Inventory management testing focused on stock quantity updates and low-stock monitoring. When product quantities were modified, the updated values were verified through both the application and database. Low-stock conditions were tested to ensure that the system could identify products requiring attention.

Order placement was tested using valid products and available quantities. A successful order was expected to create the required order record and associate it with the correct customer. The system was also tested with unavailable products and invalid quantities to verify that incorrect orders were rejected appropriately.

The order lifecycle was tested by changing order states through the supported processing stages. The tests verified that order information remained consistent as the order progressed through processing, shipping, delivery, cancellation, and other supported states. The updated status was checked on both the backend and frontend.

Customer engagement and loyalty functionality was tested by performing eligible transactions and verifying the associated customer points or rewards information. The purpose of these tests was to ensure that customer-related information was correctly connected with order activity and that the resulting information could be displayed to the

user.

Notification testing was performed by triggering supported application events and checking whether the corresponding notification information was generated and displayed correctly. Duplicate or incorrect notifications were also considered during validation.

Return management was tested using eligible orders and valid return information. The system was expected to associate the return with the correct order and maintain the appropriate return status. Invalid order references and unsuitable return requests were tested to verify that the system responded correctly.

Analytics testing verified whether the dashboard correctly represented data related to sales, revenue, order trends, product performance, inventory, and returns. The values returned by backend APIs were compared with the information displayed through Chart.js visualisations. This ensured that the analytical dashboard was connected correctly to the underlying transactional data.

The Business Decision Support module was tested using available sales, inventory, and return information. The objective was to verify that the module could process relevant business data and provide useful insights for decision-making. The results were checked for consistency with the information available from the analytics and database modules.

Database testing was performed throughout the application. After successful operations, the corresponding MySQL records were checked to ensure that data had been correctly inserted or updated. Relationships between users, products, orders, inventory, and returns were also considered to verify data integrity.

Integration test cases were executed to validate complete workflows involving multiple modules. For example, a customer could log in, select a product, place an order, trigger inventory changes, receive order-related information, and later view the updated order status. Such complete workflows helped confirm that the different application modules communicated correctly.

Security test cases included attempts to access protected APIs without authentication, with invalid JWT tokens, and with insufficient user privileges. These tests confirmed that restricted operations were protected and that unauthorised requests were handled appropriately.

Frontend testing verified navigation, form submission, dashboard rendering, order information, inventory information, customer features, and analytics visualisations. The application was also tested for appropriate error messages when backend services returned invalid or unsuccessful responses.

The final testing process demonstrated that the major functionalities of the Digital Order Management and Retail Analytics Portal operated correctly under the tested conditions. Minor issues identified during development were corrected and the related test cases were executed again to confirm successful resolution. The repeated execution of test cases also helped ensure that modifications to one module did not unintentionally affect previously implemented

functionality.

Overall, the testing results confirmed the functional integration of the React.js frontend, Node.js and Express.js backend, Sequelize ORM, and MySQL database. The system successfully supported its major objectives of digital order management, inventory monitoring, customer engagement, analytics, and business decision support. The testing process therefore provided confidence in the reliability, security, consistency, and usability of the Digital Order Management and Retail Analytics Portal and established that the application was suitable for demonstration and further deployment.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Digital Order Management and Retail Analytics Portal represents the final stage of the software development lifecycle, transforming the developed application from a local development system into an accessible and operational web-based platform. Deployment is an important phase because it ensures that the frontend, backend, database, authentication mechanisms, order management features, inventory services, customer management, loyalty functionality, analytics dashboard, and decision support components work together in a stable environment. The deployment process requires proper preparation of the application, configuration of the runtime environment, database setup, backend service configuration, frontend build generation, security configuration, and final verification of all major functionalities.

The initial step in deployment involves preparing the project environment. The development team ensures that the latest stable version of the source code is available through the Git repository. Before deployment, the React.js frontend and Node.js with Express.js backend are checked to ensure that all required dependencies are correctly installed and that the application runs successfully in the development environment. The project dependencies are maintained through package.json and package-lock.json files. The frontend is prepared for production using the appropriate npm build process, which generates optimised static files containing the HTML, CSS, JavaScript, and other required assets. The backend is prepared as a Node.js application with Express.js responsible for handling API requests and communication with the database.

The next step involves configuring the MySQL database required by the application. The production database is created separately from the development database to avoid accidental modification or deletion of development data. The required tables and relationships are created according to the database design implemented through Sequelize ORM. The database contains information related to users, customer profiles, products, orders, order items, inventory, returns, notifications, loyalty points, and other supporting entities required by the system. Sequelize is used to establish communication between the Node.js backend and MySQL database. Before the application is made available to users, database connectivity and important CRUD operations are verified to ensure that data can be created, retrieved, updated, and deleted correctly.

After the database is prepared, the backend service is configured for deployment. The Node.js and Express.js application is started using the production environment configuration. Environment variables are configured to store important information such as the MySQL database connection details, JWT secret key, backend port, and frontend API configuration. Sensitive information is not stored directly inside the source code. Instead, environment variables are used so that configuration values can be changed between development and production without modifying application logic. The backend API is then tested to ensure that authentication, user management, product operations, order processing, inventory management, notifications, returns, loyalty operations, analytics data retrieval, and decision support services respond correctly.

The frontend deployment process involves generating a production build of the React.js application. The production build contains the optimised application files that can be served through a suitable web hosting service or web server.

The frontend is configured with the correct backend API URL so that requests generated by the React.js application are directed to the deployed Express.js backend. All major frontend pages, including login, registration, dashboard, products, orders, inventory, customer management, returns, loyalty, notifications, analytics, and decision support pages, are checked after deployment to ensure that navigation and API communication work correctly.

Security configuration forms an important part of the deployment process. JWT-based authentication is used to protect authenticated resources within the system. User credentials are verified during login, and authenticated requests use the generated JWT token to access protected backend APIs. Role-based access is maintained so that administrators, staff members, and customers can access only the functions appropriate to their responsibilities. Cross-Origin Resource Sharing (CORS) is configured in the Express.js backend so that the deployed React.js frontend can communicate securely with the backend API. Database credentials, JWT secrets, and other sensitive configuration values are maintained through environment variables rather than being exposed in the source code.

Once the frontend, backend, and database have been configured, integration verification is performed. The complete order lifecycle is tested beginning from customer login and product selection through order placement, processing, shipping, delivery, cancellation, and return operations. Inventory updates are checked to ensure that product quantities are correctly reflected after orders and returns. Customer loyalty points and rewards are verified to ensure that customer engagement information is updated correctly. Notifications are also checked to confirm that important system events are communicated to the appropriate users.

The analytics and decision support modules are verified after the main application services are operational. The analytics dashboard retrieves relevant sales, revenue, order, product, inventory, and return information from the backend and presents it using Chart.js visualisations. The Business Decision Support module uses available business data to provide useful insights for administrators. The deployed application is checked to ensure that these modules receive current database information and display meaningful results without errors.

After successful deployment, post-deployment verification is carried out across the complete application. User registration and login, role-based access, product management, order placement, inventory monitoring, customer profiles, loyalty points, notifications, return management, analytics, and decision support are tested. API responses are verified using development and testing tools, while browser-based verification is performed to ensure that the user interface displays correctly. The application is also checked for broken links, incorrect API endpoints, authentication errors, database connection failures, and unexpected behaviour.

Finally, deployment documentation and version control records are maintained for future maintenance. The deployed version is identified through the Git repository so that changes can be tracked and previous stable versions can be identified if rollback becomes necessary. Database backups and configuration records are maintained to support recovery in case of unexpected system failure. Any future modification to the system should follow the same development, testing, and deployment process to maintain the reliability and consistency of the application.

In summary, the deployment of the Digital Order Management and Retail Analytics Portal is a structured process involving application preparation, database configuration, backend deployment, frontend build generation, security configuration, API integration, and post-deployment verification. By following these steps, the system can provide a

stable environment for managing digital orders, inventory, customers, returns, loyalty programmes, analytics, and business decision support.

9.2 Environment Configuration

Environment configuration represents one of the most important aspects of deploying the Digital Order Management and Retail Analytics Portal, as it provides the foundation required for the React.js frontend, Node.js and Express.js backend, Sequelize ORM, MySQL database, JWT authentication, and Chart.js analytics components to operate together. A properly configured environment ensures that dependencies, database connections, authentication services, API communication, and application settings remain consistent throughout development and deployment. The configuration process also helps prevent errors caused by missing dependencies, incorrect database credentials, incompatible software versions, or incorrect API URLs.

The development environment consists primarily of the software tools required to build and test the application. The frontend is developed using React.js and provides the user interface through which customers, staff, and administrators interact with the system. Chart.js is integrated into the frontend to generate graphical representations of sales, revenue, order trends, product performance, and return-related information. The backend is developed using Node.js and Express.js, which provide the runtime environment and server-side framework for handling HTTP requests, implementing API endpoints, performing authentication, and communicating with the database.

The database environment is configured using MySQL Server. MySQL stores the persistent information required by the portal, including user details, customer information, products, orders, order items, inventory information, return records, notifications, loyalty information, and analytical data. Sequelize ORM acts as the database abstraction layer between the Node.js backend and MySQL. It provides models and database operations that allow the application to work with relational data without placing all database operations directly inside individual frontend components. Sequelize also supports structured database relationships and helps maintain consistency between the application models and database tables.

Dependency management is handled through npm and the package.json file. All required frontend and backend packages are declared as project dependencies so that the development environment can be reproduced when the project is installed on another system. The package-lock.json file maintains the installed dependency versions and helps reduce differences between development and deployment environments. Node.js provides the runtime required for the backend, while npm is used for installing dependencies and executing development and production commands.

Environment variables are used to manage configuration values that should not be hard-coded into the application. These variables may include the MySQL host, database name, database username, database password, backend port, JWT secret key, and frontend API base URL. During development, these values can be maintained in a local environment configuration file. Sensitive configuration files are excluded from version control through the project's .gitignore configuration. In a production environment, the corresponding variables are configured through the hosting environment so that confidential information is not exposed in the source code repository.

The authentication environment is configured around JSON Web Tokens. JWT provides secure authentication for

users after successful login. The backend verifies the submitted credentials and generates a token that is used when accessing protected API endpoints. The frontend stores and sends the token as required for authenticated requests. The backend validates the token before allowing access to protected resources. Role-based authorisation is also maintained so that administrators, staff, and customers receive appropriate access to the application's features.

The frontend environment is configured to communicate with the Express.js backend through API requests. The API base URL is configured according to the deployment environment. During local development, the frontend communicates with the locally running backend service, while the production frontend is configured to communicate with the deployed backend API. This separation makes it possible to deploy the same frontend application in different environments without changing the core application logic.

The Express.js backend environment contains the server configuration, routes, middleware, authentication mechanisms, business logic, and Sequelize database connection. Express middleware is used to process incoming requests, manage authentication, handle CORS, and support appropriate API behaviour. The backend is configured to listen on the required server port and establish a connection with the MySQL database when the application starts. Database connection errors are handled so that configuration problems can be identified during deployment and maintenance.

The MySQL environment requires correct database credentials and appropriate database permissions. The database user configured for the application should have only the permissions required for normal application operations. The database should also be backed up regularly to protect important order, customer, inventory, and business data. Database tables and relationships are maintained according to the Sequelize models used by the backend. This configuration allows the application to maintain consistent data between different modules.

The testing environment is configured separately from production data whenever possible. Test users, products, orders, and other sample records can be used to verify application functionality without affecting real operational information. API endpoints are tested independently and as part of complete workflows. Browser Developer Tools can be used to inspect frontend requests, responses, console errors, and network communication between the React.js frontend and Express.js backend.

The development tools used for maintaining the environment include Visual Studio Code for source-code development and Git for version control. Git allows the development team to maintain different versions of the application and track modifications made to the frontend, backend, and configuration files. The repository contains source code and project configuration, while sensitive information such as database passwords and secret keys is excluded from version control.

Security is maintained throughout environment configuration by following the principle of protecting sensitive configuration values and limiting access to system resources. JWT secrets, database passwords, and other confidential values should not be included directly within source files. CORS should be configured to permit communication from trusted frontend origins. Dependencies should also be reviewed and updated when required to reduce security risks associated with outdated packages.

The environment configuration also supports maintainability by keeping the frontend, backend, and database responsibilities clearly separated. React.js handles presentation and user interaction, Node.js and Express.js handle server-side processing and API services, Sequelize provides structured database interaction, MySQL stores persistent data, JWT manages authentication, and Chart.js provides analytical visualisation. This separation allows individual components to be maintained or improved without unnecessarily changing the complete application.

In conclusion, the environment configuration of the Digital Order Management and Retail Analytics Portal establishes a consistent and secure foundation for the complete application. Through proper configuration of React.js, Node.js, Express.js, Sequelize ORM, MySQL, JWT, Chart.js, npm dependencies, environment variables, and version control, the system can be developed, tested, and deployed in a controlled manner. This structured configuration reduces deployment errors, improves maintainability, and supports reliable operation of the digital order management and retail analytics system.

9.3 Hosting of Frontend and Backend

Hosting represents the stage in which the Digital Order Management and Retail Analytics Portal becomes accessible outside the local development environment. The hosting architecture must support the React.js frontend, Node.js and Express.js backend, and MySQL database while maintaining secure communication between the different components. The hosting arrangement is designed to ensure that customers, staff, and administrators can access the application through a web browser and that all frontend operations can communicate correctly with the backend API and database.

Frontend Hosting

The frontend of the portal is developed using React.js and is prepared for hosting by generating a production build. The production build contains the optimised static files required to run the web application. These files can be hosted using a suitable web hosting platform or web server capable of serving React.js applications. The hosting environment must be configured with the correct production API URL so that frontend requests are directed to the deployed Node.js and Express.js backend.

The hosted React.js application provides the user interface for all major system operations. Customers can use the frontend to browse products, place orders, view order information, manage their profiles, receive notifications, and participate in the loyalty programme. Staff and administrators can access the appropriate management functions based on their assigned roles. The analytics dashboard displays business information through Chart.js, while the decision support module presents useful insights based on available sales, inventory, order, and return data.

The frontend hosting environment should support HTTPS to protect communication between the user's browser and the application. Static assets such as JavaScript files, CSS files, images, and other frontend resources are served through the hosting platform. Browser caching can also improve performance by reducing the need to download unchanged resources repeatedly. The production frontend is updated whenever a new stable version of the application is built and deployed.

Backend Hosting

The backend is developed using Node.js and Express.js and is responsible for providing REST-based API services to the React.js frontend. The backend can be hosted on a suitable cloud server or Node.js-compatible hosting platform. The hosting environment must provide the required Node.js runtime and allow the Express.js application to listen for incoming HTTP requests.

The deployed backend manages authentication, user roles, product operations, order processing, inventory management, customer engagement, loyalty points, notifications, returns, analytics data retrieval, and decision support operations. It communicates with the MySQL database through Sequelize ORM. Whenever a user performs an operation through the frontend, the corresponding API request is sent to the Express.js backend. The backend validates the request, performs the required business operation, communicates with MySQL when necessary, and returns the appropriate response to the frontend.

The backend hosting environment is configured using environment variables for database credentials, JWT secrets, application ports, and other deployment-specific settings. Sensitive information is not included in the application source code. The server should also use HTTPS communication and appropriate CORS configuration to ensure that only authorised frontend origins can communicate with the API.

Database Hosting

The MySQL database can be hosted on a dedicated database server or a managed cloud database service. The database stores the persistent information generated by the portal, including user records, products, inventory quantities, orders, order items, customer information, returns, notifications, loyalty records, and other system data.

The backend connects to the hosted MySQL database using Sequelize ORM. The database connection is configured through environment variables so that credentials are not exposed in the source code. Database access is restricted to the backend service rather than being directly exposed to frontend users. Regular backups are recommended to protect important business information and support recovery in the event of database failure or accidental data loss.

Frontend–Backend Communication

The React.js frontend and Node.js backend communicate through API endpoints. When a user performs an action such as logging in, placing an order, checking inventory, viewing notifications, or accessing analytics, the frontend sends an HTTP request to the appropriate Express.js API endpoint. The backend processes the request and communicates with MySQL through Sequelize ORM when database information is required.

JWT authentication is used for protected operations. After successful authentication, the frontend includes the JWT token with requests to protected endpoints. The backend validates the token and determines the user's role before allowing access to the requested resource. This architecture ensures that sensitive operations such as administrative product management, inventory updates, order management, and analytics access are protected from unauthorised users.

Deployment and Maintenance

The hosting environment requires continuous maintenance after the initial deployment. New versions of the frontend

and backend are first tested in the development environment before being deployed to the hosted system. Git is used to maintain source-code versions and track modifications. When a new stable version is available, the production frontend build and backend application are updated accordingly.

Monitoring of the hosted application includes checking server availability, API response errors, database connectivity, authentication failures, and application behaviour. Any unexpected issue is investigated through backend logs, browser Developer Tools, database verification, and API testing. Database backups are maintained to reduce the impact of potential data loss, while source-code version control allows previous stable versions to be identified when troubleshooting deployment problems.

The hosting architecture also supports future scalability. As the number of customers, orders, products, and analytical records increases, the frontend and backend hosting resources can be upgraded according to system requirements. The database can also be optimised through appropriate indexing, query optimisation, and resource management. Since the application is divided into frontend, backend, and database layers, each layer can be improved independently without requiring a complete redesign of the system.

Post-Deployment Verification

Following deployment, a complete verification process is performed to ensure that the hosted system operates correctly. User registration and login are tested first, followed by role-based access verification. Product browsing, product management, order placement, order status updates, cancellation, returns, inventory updates, customer profiles, loyalty points, notifications, analytics, and decision support features are then tested.

The communication between the React.js frontend and Express.js backend is verified through actual API requests. Database operations are checked to confirm that records are correctly stored and retrieved from MySQL. The analytics dashboard is verified to ensure that Chart.js receives the expected information and displays the corresponding visualisations. The Business Decision Support module is also checked to ensure that its results are generated from available business information.

In conclusion, the hosting of the Digital Order Management and Retail Analytics Portal provides the infrastructure required to make the application accessible to its intended users. The React.js frontend provides the interactive user interface, the Node.js and Express.js backend provides API and business services, Sequelize ORM manages communication with MySQL, and the MySQL database provides persistent storage. JWT protects authenticated operations, while Chart.js supports the presentation of retail analytics. Together, these hosted components provide a complete environment for digital order management, inventory control, customer engagement, analytics, and business decision support.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The development of the Digital Order Management and Retail Analytics Portal presented a range of technical, functional, database, integration, and coordination challenges throughout the software development lifecycle. Since the proposed system combines order management, inventory monitoring, customer engagement, loyalty management, notifications, analytics, and business decision support within a single web-based platform, the development process required careful coordination between the frontend, backend, database, authentication, and analytics components. The major challenges were encountered during system design, database implementation, API development, frontend integration, authentication, data processing, testing, and overall system integration. Each challenge provided an opportunity to understand practical software development problems and improve the reliability and usability of the final application.

One of the initial challenges involved designing a suitable database structure for managing different types of retail information. The system needs to maintain relationships between users, customers, products, inventory records, orders, order items, payments, returns, notifications, loyalty points, and other related information. Designing these relationships while avoiding unnecessary data duplication required careful planning. During the early development stages, changes in one database entity occasionally affected related tables and API operations. Maintaining proper primary keys, foreign keys, relationships, and constraints was therefore an important part of the database design process.

Another significant challenge was implementing the complete order lifecycle. An order does not remain in a single state; it moves through different stages such as placement, processing, shipping, delivery, cancellation, and return. Ensuring that these status changes were correctly reflected in the database and frontend interface required careful implementation of backend logic. Incorrect status transitions could result in inconsistent information being displayed to users. The system therefore required appropriate validation and controlled update operations so that order information remained consistent throughout its lifecycle.

Inventory management also presented several development challenges. The system needs to maintain product stock information while orders are being processed. When an order is placed, the corresponding inventory quantity must be updated correctly. Low-stock conditions must also be identified so that administrators or staff members can take appropriate action. Managing inventory together with order processing required careful coordination between database transactions and backend API operations. Incorrect stock updates could result in inaccurate availability information or inconsistencies between product and order data.

The frontend development phase introduced challenges related to handling dynamic data from multiple backend APIs. React.js provides a component-based development approach, but maintaining consistent application state across dashboards, product information, orders, customer profiles, inventory records, notifications, and analytics required careful organisation. API responses could arrive asynchronously, and improper handling of loading states or errors could result in incomplete information being displayed. Designing a user interface that remained

understandable while presenting different types of retail information was also an important challenge.

Another challenge involved integrating the React.js frontend with the Node.js and Express.js backend. The frontend and backend operate as separate layers and communicate through API requests. Correct API endpoints, request methods, request data, response structures, and authentication headers had to be maintained consistently. During development, mismatches between frontend expectations and backend responses could lead to errors in displaying or updating information. Debugging these issues required testing individual API endpoints and verifying the complete communication process between both layers.

Authentication and role-based access control also required careful implementation. The portal provides different access levels for administrators, staff, and customers. Each role should have access only to the functions relevant to that role. Implementing JWT-based authentication required secure token generation, token validation, protected API routes, and appropriate frontend handling of authenticated sessions. Incorrect role validation could potentially allow users to access functions that should be restricted. Therefore, authentication and authorisation were treated as important parts of the system implementation.

Analytics development presented another challenge because the dashboard needs to convert database information into meaningful visual representations. Sales, revenue, order trends, product performance, and return patterns need to be calculated from the available system data before they can be displayed using Chart.js. The challenge was not only generating charts but also ensuring that the underlying calculations were correct. Incorrect aggregation or filtering could produce misleading charts and affect the usefulness of the analytics dashboard.

The Business Decision Support module introduced additional challenges because it combines information from different areas of the system. Sales data, inventory information, order activity, and return patterns need to be considered together to generate useful business insights. Designing the logic so that recommendations were based on meaningful data rather than isolated values required careful consideration. The module had to provide understandable recommendations that could support administrative decision-making without unnecessarily complicating the user interface.

The Sequelize ORM also introduced challenges during database integration. Sequelize provides an abstraction layer between Node.js and MySQL, but model definitions, relationships, queries, and database operations still needed to be configured correctly. Changes to models could affect associated operations and API responses. Debugging database queries sometimes required checking both the Sequelize implementation and the underlying MySQL tables to determine whether the problem originated in application logic or database structure.

Testing and debugging represented another major area of difficulty. Individual features could work correctly when tested separately but produce unexpected behaviour when integrated with other modules. For example, an order operation could affect inventory, customer loyalty points, notifications, and analytics data simultaneously. Testing such interconnected operations required multiple scenarios and repeated verification. API testing using Postman was particularly useful for checking request and response behaviour before connecting the APIs to the frontend.

Data validation was also an important challenge. User inputs such as registration information, product details, order information, quantities, and other form values need to be validated before being stored in the database. Invalid or incomplete data could cause backend errors or produce incorrect analytics. Both frontend and backend validation had to be considered so that the system could provide meaningful error messages and maintain reliable data.

Another challenge involved notification management. Notifications need to be generated for relevant activities such as order updates, inventory conditions, or other system events. Ensuring that notifications were created at the appropriate stage without generating unnecessary duplicate records required careful backend logic. The notification interface also needed to display information clearly so that users could understand the reason and status of each notification.

The loyalty and rewards functionality introduced additional challenges related to maintaining accurate customer points. Points need to be associated with the correct customer and updated according to the defined business rules. Any incorrect calculation could affect customer rewards and reduce confidence in the system. Therefore, loyalty-related operations required validation and testing using different customer and order scenarios.

Finally, team coordination and version control created challenges during collaborative development. Different components of the application were developed and modified at different stages, which could lead to conflicts when combining changes. Maintaining consistent naming conventions, API structures, database models, and frontend component organisation was necessary to reduce integration problems. Git and GitHub were useful for maintaining version history and coordinating development activities among team members.

In summary, the development of the Digital Order Management and Retail Analytics Portal was an iterative process involving several technical and organisational challenges. The difficulties experienced during database design, API integration, authentication, inventory management, analytics, frontend development, testing, and team coordination contributed to a deeper understanding of real-world software development. Resolving these issues helped improve the reliability, consistency, usability, and maintainability of the final retail management system.

10.2 Solutions Applied

The challenges encountered during the development of the Digital Order Management and Retail Analytics Portal were addressed through systematic improvements to the database, backend, frontend, authentication, analytics, and testing processes. Rather than treating individual problems as isolated issues, the development process focused on maintaining proper communication between the different layers of the application. The solutions applied helped improve system reliability, data consistency, security, usability, and overall operational efficiency.

To address the database design challenges, the system was organised using a structured relational database approach with MySQL. Separate entities were created for major functional areas such as users, customers, products, inventory, orders, order items, returns, notifications, and loyalty information. Relationships between related entities were maintained using appropriate keys and associations. Sequelize ORM was used to define and manage these database models from the Node.js backend. This approach reduced unnecessary duplication and

provided a structured method for accessing and updating database information.

For order lifecycle management, controlled backend operations were implemented for different order statuses. Instead of allowing arbitrary status changes, the application logic validates the requested operation before updating the order. The system maintains the order state consistently between the database and frontend. API operations were also tested with different scenarios such as successful orders, cancellations, deliveries, and returns to ensure that status changes were correctly reflected throughout the application.

Inventory-related issues were addressed by connecting product and inventory operations with order processing. Stock quantities are updated when relevant order operations are completed, and low-stock conditions can be identified for administrative monitoring. Backend validation was introduced to reduce the possibility of invalid quantities or incorrect stock updates. Database operations through Sequelize were used to ensure that inventory information remained connected to the corresponding product records.

Frontend state and API communication issues were addressed through a structured React.js component architecture. Individual application features were divided into manageable components and pages so that responsibilities remained clear. API requests were organised according to the functionality of the system, and loading and error states were handled appropriately. This reduced the possibility of displaying incomplete information and made individual features easier to test and maintain.

To solve frontend-backend communication problems, REST-based API integration was standardised using Node.js and Express.js. Each operation was associated with an appropriate API endpoint, request method, and response structure. The frontend sends requests to the backend, while the backend performs the required business logic and database operations before returning the response. Postman was used during development to test APIs independently, making it easier to identify whether errors originated from the frontend or backend.

Authentication and authorisation challenges were addressed using JWT-based authentication. The backend generates authentication tokens for valid users, while protected routes verify the token before allowing access to restricted resources. Role-based access control was applied to distinguish administrators, staff, and customers. This ensured that sensitive administrative operations were not available to users without the required permissions. Passwords and authentication information were handled through secure backend practices, while frontend requests included the required authentication information when communicating with protected APIs.

Sequelize ORM helped simplify the interaction between the Express.js backend and MySQL database. Instead of writing every database operation directly as raw SQL, Sequelize models and queries were used to manage application data. Associations between models were configured so that related information could be retrieved efficiently. Database errors were investigated by checking both Sequelize model definitions and the underlying MySQL structure, allowing problems to be isolated more effectively.

The analytics dashboard challenges were addressed by creating backend data calculations and presenting the resulting information through Chart.js. Important indicators such as sales, revenue, order trends, product performance, and return patterns were processed from system data. The dashboard then presents this information using suitable charts and visual elements. This approach allows administrators to understand business performance

more easily than by examining raw database records.

For the Business Decision Support module, information from sales, inventory, orders, and returns was considered together to identify useful business patterns. Instead of displaying only raw statistics, the module was designed to convert available information into understandable insights. For example, sales and inventory information can help identify products requiring attention, while return information can help highlight products or order patterns that may require further review. This provides a more practical use of the analytics data for business decision-making.

Customer loyalty issues were addressed by associating loyalty points with the appropriate customer records and applying defined business rules during relevant order operations. Point calculations and updates were tested using different scenarios to ensure that customer information remained consistent. The loyalty system was also integrated with customer engagement functionality so that customers could view their rewards-related information through the portal.

Notification-related issues were addressed by implementing notification creation and retrieval through backend APIs. Notifications are associated with relevant system activities and presented through the frontend interface. Validation was added to reduce unnecessary or duplicate notification generation. This allows customers, staff, and administrators to receive information relevant to their activities within the system.

Input validation was improved across both frontend forms and backend APIs. Required fields, valid values, quantities, and other relevant information were checked before processing. Clear error responses were provided when invalid data was submitted. This reduced the possibility of incorrect information entering the database and improved the overall user experience by informing users about problems with their inputs.

Testing and debugging were improved through a layered testing approach. Individual API endpoints were tested using Postman, database records were verified through MySQL, and frontend behaviour was tested through the React.js application. Integration testing was then performed to verify that multiple modules worked correctly together. Scenarios involving orders, inventory, customers, notifications, loyalty points, and analytics were tested to ensure that changes in one part of the system did not produce unexpected results elsewhere.

Chart.js-related issues were addressed by verifying the data supplied to each chart before rendering it. Appropriate labels, values, and categories were checked to ensure that the visualisations accurately represented the underlying data. Dashboard calculations were also compared with database records during testing so that incorrect aggregation or filtering could be identified before final deployment.

Version control and team coordination challenges were addressed through Git and GitHub. Development changes were maintained through version control so that previous versions could be reviewed when necessary. Team members could work on different components and integrate their changes systematically. Consistent project structure, naming conventions, and communication helped reduce conflicts and made the development process more organised.

The overall development process was also improved through repeated testing and debugging. Problems were first identified through API or application testing, then traced to the relevant frontend, backend, database, or integration layer. After applying a correction, the affected functionality was tested again to ensure that the solution did not introduce additional problems. This iterative approach helped improve the stability of the complete application.

In conclusion, the solutions applied during development addressed the major technical and functional challenges of the Digital Order Management and Retail Analytics Portal. The combination of React.js, Node.js, Express.js, Sequelize ORM, MySQL, JWT authentication, Chart.js, Postman, Git, and GitHub provided a structured development environment. The corrective measures applied to database management, order processing, inventory control, authentication, analytics, customer engagement, notifications, and testing helped transform the initial implementation into a more reliable and integrated retail management platform.

10.3 Current Limitations or Known Bugs

Although the Digital Order Management and Retail Analytics Portal provides a wide range of functionality for retail order management, inventory monitoring, customer engagement, analytics, and decision support, certain limitations remain within the current implementation. These limitations are mainly associated with the project scope, development time, available resources, integration complexity, and the current level of automation. The system successfully demonstrates its major objectives, but several areas can be further enhanced in future versions.

One of the main limitations is that the system currently depends on the data entered and maintained within its own database. The accuracy of analytics, inventory information, order information, and business recommendations depends on the quality and completeness of the stored data. If incorrect product quantities, prices, order details, or return information are entered, the resulting analytics may also be inaccurate. Therefore, stronger validation and automated data verification can be introduced in future versions.

Another limitation is related to inventory synchronisation during multiple simultaneous orders. When several users attempt to purchase the same product at nearly the same time, maintaining accurate stock quantities can become more complex. Although the current system performs inventory updates through backend operations, high-concurrency scenarios may require stronger transaction handling and locking mechanisms to guarantee complete consistency under heavy traffic.

The analytics dashboard also has certain limitations. The dashboard currently focuses on major retail indicators such as sales, revenue, order trends, product performance, and return patterns. More advanced analytics, including predictive sales forecasting, customer churn prediction, seasonal demand forecasting, and advanced product recommendation models, are outside the current scope. Such capabilities could be incorporated in future versions to make the analytics component more intelligent.

The Business Decision Support module also has limitations because its recommendations depend primarily on the available sales, inventory, order, and return information. The current system does not incorporate advanced machine learning models or external market information. As a result, the recommendations provide data-based support rather than fully predictive business intelligence. Future development could extend this module using

historical trend analysis and predictive techniques.

Another limitation concerns the current notification functionality. Notifications are generated based on defined application events, but the system may not provide a fully automated multi-channel notification mechanism across email, SMS, and mobile push notifications. Expanding notification delivery to multiple channels could improve communication between the portal and its users.

The loyalty and rewards functionality also has scope for future improvement. The current points-based approach provides a basic mechanism for rewarding customers, but more advanced loyalty strategies could be introduced. These may include tier-based membership, personalised rewards, promotional campaigns, discount management, and customer-specific offers. Such enhancements could make the customer engagement module more comprehensive.

The frontend may also experience limitations under low network conditions or on devices with smaller screens. Since the application depends on communication with backend APIs, slow network connections can increase loading time. Some data-intensive dashboard components may also require additional optimisation as the amount of stored data increases. Further responsive design and performance optimisation could improve the experience across a wider range of devices.

Authentication and session management may also require further enhancement for large-scale deployment. JWT authentication provides secure access control, but production environments may require additional measures such as more advanced token management, stronger password policies, account recovery mechanisms, multi-factor authentication, and detailed security monitoring. These improvements would strengthen the security of the system as the number of users increases.

The current system is also limited in terms of advanced administrative reporting. Administrators can view important analytics through the dashboard, but future versions could provide downloadable reports in formats such as PDF or Excel. Scheduled reports, custom date-range reports, department-specific reports, and automated management summaries could further improve the usefulness of the platform.

Another known limitation concerns scalability. The current architecture is suitable for the intended academic project and demonstration environment, but handling a very large number of simultaneous users, orders, and database operations would require additional infrastructure optimisation. Database indexing, caching, load balancing, background processing, and horizontal scaling could be introduced if the system were extended to a large commercial deployment.

The system currently focuses primarily on core retail order management and analytics. Integration with external services such as advanced payment gateways, third-party logistics providers, external warehouse management systems, or enterprise resource planning systems may require additional development. Such integrations would allow the portal to support a more complete commercial retail environment.

Testing coverage is another area that can be improved. Although the major modules and API operations have been tested, a larger production-level application would require more extensive automated unit, integration, regression, security, and performance testing. Continuous automated testing could help detect unexpected problems whenever new features or database changes are introduced.

The database may also experience performance limitations as the volume of historical orders, customer records, inventory information, and analytics data increases. Queries that are efficient with a small academic dataset may require further optimisation when dealing with millions of records. Additional indexing, query optimisation, database partitioning, caching, or database scaling strategies could therefore be considered for future versions. Another limitation is that the Business Decision Support module does not currently provide fully autonomous decision-making. The system presents useful information and recommendations, but the final business decision remains with the administrator or responsible staff member. This is appropriate for a decision-support system because business actions should be reviewed by authorised users rather than being performed automatically without human verification.

The project also has limitations related to deployment and infrastructure because the current implementation is primarily designed as an academic web application. The actual production environment may require additional configuration for domain management, HTTPS, server monitoring, automated backups, recovery procedures, scaling, and continuous deployment. These aspects can be expanded if the project is developed into a commercial system.

From a user experience perspective, further accessibility improvements can also be considered. Future versions can provide stronger keyboard navigation, improved accessibility labels, enhanced colour contrast, and better support for users with different accessibility requirements. These improvements would make the portal more inclusive and easier to use.

In conclusion, the Digital Order Management and Retail Analytics Portal successfully implements its major objectives of centralising retail order management, inventory monitoring, customer engagement, loyalty management, analytics, and business decision support. The identified limitations do not prevent the core system from functioning, but they represent opportunities for future enhancement. Advanced analytics, predictive decision support, stronger scalability, expanded notifications, improved security, automated testing, enhanced reporting, and additional external integrations can further strengthen the system. Recognising these limitations provides a clear direction for future development and demonstrates that the project can be extended beyond its current academic scope into a more comprehensive retail management solution.

.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Digital Order Management with Database System and Retail Analytics platform has been conceived with modularity, scalability, and long-term extensibility in mind, allowing future enhancements to be integrated seamlessly without disrupting existing core retail workflows. While the current implementation provides a robust and centralized solution for order processing, inventory monitoring, role-based security, customer engagement, and business analytics, the project's long-term vision encompasses several planned features aimed at elevating operational efficiency, automating managerial intelligence, and expanding the application's capabilities to meet modern commercial e-commerce standards.

Foremost among the planned features is the integration of advanced Machine Learning (ML) algorithms for predictive demand forecasting. The current system provides rule-based stock monitoring and threshold alerts; however, future releases will incorporate predictive time-series models (such as ARIMA, Prophet, or Random Forest regressors) trained on historical sales patterns, seasonal demand fluctuations, promotional schedules, and supplier lead times. This enhancement will allow store managers and administrators to anticipate stock shortages weeks in advance, automatically calculate optimal reorder points, and generate automated purchase recommendations to suppliers, thereby minimizing holding costs while preventing lost revenue from stockouts.

Another critical enhancement planned for the storefront is an intelligent, personalized product recommendation engine. By analyzing customer browsing sessions, previous order histories, wishlist interactions, and cart additions, collaborative filtering and content-based recommendation models can be applied. When customers view a product or open their dashboard, the platform will suggest contextually relevant products, complementary accessories, or trending items. This personalized shopping experience directly improves customer engagement, enhances cross-selling and up-selling opportunities, and increases the average order value across the retail platform.

The loyalty and rewards component will be expanded into a comprehensive multi-tier loyalty and gamification program. While the current implementation provides a reliable points-based mechanism, future iterations will feature structured membership tiers such as Silver, Gold, and Platinum. Customers will unlock exclusive tier benefits, such as accelerated points accumulation, member-only discounts, early access to new product releases, personalized anniversary vouchers, and referral incentives. The system will also introduce gamified milestones and streak bonuses, encouraging sustained customer engagement and brand loyalty.

To enhance customer communication and transparency, the notification subsystem will be upgraded into an automated multi-channel communication matrix. By integrating enterprise communication APIs such as Twilio for SMS and WhatsApp messaging, alongside SendGrid or AWS Simple Email Service (SES) for transactional emails, customers will receive real-time dispatch alerts, digital invoice attachments, and live courier tracking links directly on their preferred messaging channels. Operational staff will also benefit from automated low-stock warnings and critical order alerts dispatched via instant messaging channels.

Another significant future addition is the development of a dedicated native mobile application for both iOS and Android platforms using React Native. While the current web portal is fully responsive across desktop and tablet viewports, a dedicated mobile app will offer camera-based barcode and QR code scanning for warehouse personnel to expedite inventory audits, barcode printing, and package dispatch verification. For retail customers, the mobile application will provide frictionless one-touch biometric authentication, push notifications for flash sales and delivery tracking, and offline browsing capabilities.

Furthermore, the administrative module will be augmented with comprehensive export and reporting tools. Business executives frequently require downloadable reports for external audits, tax filing, and corporate board meetings. Future updates will introduce one-click export capabilities for financial ledgers, inventory valuation statements, customer churn analytics, and tax compliance summaries in formatted PDF and Microsoft Excel (.xlsx) formats, complete with customizable date ranges and department-level filters.

11.2 Possible Integrations or Optimisations

As the Digital Order Management and Retail Analytics Portal scales to accommodate larger product catalogs, higher transaction volumes, and concurrent administrative users, targeted technical optimizations and external integrations will be introduced to maintain high throughput, low latency, and robust fault tolerance.

A primary architectural optimization involves decomposing the current monolithic Express.js backend into a microservices-based architecture. Decoupling functional domains—such as User Authentication, Order Processing, Inventory Management, Payment Processing, and the Analytics Engine—into independent containerized services orchestrated via Kubernetes will provide granular scalability and fault isolation. If analytical query traffic surges during peak business review periods, the analytics service can scale horizontally across multiple worker nodes without consuming resources required for processing high-priority live customer orders.

To enhance read performance and reduce load on the primary relational database, a distributed in-memory caching layer utilizing Redis will be implemented between the Node.js application and MySQL Server. Frequently accessed static and semi-static datasets—such as active product listings, category trees, storewide promotions, and summarized dashboard analytics—will be cached in Redis with dynamic invalidation policies. This caching mechanism will significantly reduce repeated database hits, lower round-trip latency, and deliver sub-millisecond response times for storefront browsing.

Asynchronous job queues and event-driven messaging brokers, such as RabbitMQ or Apache Kafka, represent another major integration priority. Resource-intensive tasks—such as batch order processing, PDF invoice rendering, periodic sales aggregation, and multi-channel notification dispatches—will be offloaded from the main HTTP execution thread into background worker queues. This decoupling guarantees that customer checkout requests and administrative operations remain responsive, regardless of concurrent system workloads.

Database performance will be further optimized through master-slave replication and database sharding. Read-

heavy analytical operations will be directed to dedicated read-replicas, preventing complex analytical queries from locking transactional tables used during customer checkouts. In addition, implementing table partitioning on historical order records, query indexing optimizations, and connection pooling fine-tuning will ensure persistent database responsiveness even when managing millions of customer order lines.

On the client-server communication layer, the introduction of a GraphQL API alongside existing RESTful endpoints is planned. GraphQL will enable frontend components to query only the specific data fields required for rendering, eliminating over-fetching and under-fetching of data. This is particularly advantageous for mobile interfaces operating under constrained cellular bandwidth conditions.

Finally, commercial integration with third-party logistics (3PL) providers—such as FedEx, DHL, BlueDart, or Shiprocket—and ERP systems (such as SAP or Microsoft Dynamics) will bridge the gap between digital order processing and physical supply chain execution. Webhooks will automatically update delivery statuses, print shipping labels, and synchronize warehouse stock movements in real time, delivering a fully automated enterprise-grade commerce ecosystem.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Digital Order Management with Database System and Retail Analytics project represents a comprehensive web-based software engineering endeavor designed to address the persistent fragmentation, inefficiency, and lack of cohesion in traditional retail management. In legacy operational environments, businesses often rely on disparate tools—manual order entry sheets, standalone inventory databases, disconnected billing software, and isolated spreadsheet records for business reporting. This disjointed operational paradigm leads to data redundancy, delayed order fulfillment, frequent stockouts, and poor managerial visibility into sales health. The primary objective of this project was to overcome these limitations by conceptualizing, designing, and implementing a unified, data-driven platform that integrates all core retail operations into a centralized, single-window system.

The system architecture successfully bridges transactional commerce and analytical decision support. Built upon a decoupled, three-tier model, the platform combines an intuitive and responsive frontend developed with React.js, a high-performance asynchronous backend powered by Node.js and Express.js, and a robust relational database layer implemented using MySQL Server with Sequelize ORM. Security and access control are strictly enforced via stateless JSON Web Token (JWT) authentication and Role-Based Access Control (RBAC), providing tailored functional experiences for Administrators, Staff members, and Customers.

Functionally, the platform encompasses five primary interconnected modules: User Access and Role Management, Core E-Commerce Operations, Customer Engagement and Loyalty, Analytics and Admin Dashboard, and the Decision Support System (DSS). Together, these components manage the entire lifecycle of retail operations—from product listing and live stock tracking to checkout, automated inventory deductions, multi-stage order tracking, customer loyalty accumulation, return processing, and real-time visual analytics. By converting operational transaction data into actionable business intelligence through Chart.js visualizations and algorithmic inventory evaluations, the platform fulfills its mission of transforming traditional retail administration into an intelligent, streamlined, and data-centric enterprise solution.

12.2 What Was Achieved

The development and deployment of the Digital Order Management and Retail Analytics Portal achieved all predefined functional, architectural, and educational objectives established at the project's inception:

1. **Centralized Retail Management Ecosystem:** Successfully consolidated product catalog management, shopping cart operations, order processing, inventory monitoring, return workflows, and customer profiles into a single web application, eliminating the need for fragmented manual records.
2. **Full-Stack Modern Web Implementation:** Engineered a modular, component-based user interface using React.js that communicates seamlessly with a scalable RESTful API layer developed with Node.js and Express.js, ensuring high performance, responsive rendering, and smooth state management.

3. **Robust Relational Database Engineering:** Designed and normalized a MySQL database schema enforcing strict referential integrity across Users, Products, Categories, Orders, OrderItems, InventoryLogs, Returns, Notifications, and LoyaltyLedgers. Sequelize ORM was utilized to manage models, automated migrations, and complex relational associations efficiently.
4. **Complete Order Lifecycle Automation:** Successfully implemented dynamic state transitions for orders (Pending -> Processing -> Shipped -> Delivered -> Cancelled/Returned), ensuring that inventory quantities are automatically adjusted upon order placement, cancellation, and return approvals.
5. **Interactive Business Analytics Dashboard:** Integrated Chart.js to translate raw database records into clear, interactive visualizations—including sales revenue bar charts, monthly order volume trends, top-selling product breakdowns, and return rate analyses—providing management with instant visibility into business health.
6. **Algorithmic Decision Support:** Built algorithmic decision-support logic that cross-references sales velocity against current stock balances to automatically classify items as fast-moving, slow-moving, low-stock, or high-return, providing actionable restocking and pricing recommendations.
7. **Enterprise-Grade Security Architecture:** Implemented bcrypt password hashing, stateless JWT session tokens, route-level authorization middleware, parameterized query protection against SQL injection, and Cross-Origin Resource Sharing (CORS) configurations to ensure robust data security.
8. **Systematic Quality Assurance & Validation:** Conducted exhaustive functional, integration, boundary, and security testing using Postman collections and browser debugging tools, confirming that all endpoints handle valid, invalid, and edge-case requests gracefully with appropriate HTTP status codes.

12.3 Skills Learned During Development

The conceptualization, implementation, testing, and documentation of the Digital Order Management and Retail Analytics platform provided an invaluable opportunity to bridge theoretical computer science principles with practical software engineering execution. The project facilitated substantial growth across diverse technical, architectural, and collaborative domains:

- **Frontend Engineering:** Developed advanced proficiency in React.js, mastering single-page application (SPA) architecture, functional component design, custom hooks, centralized state handling, conditional UI rendering based on user roles, dynamic form validation, and asynchronous API communication using Axios.
- **Server-Side Architecture:** Acquired extensive hands-on expertise in Node.js and Express.js, gaining deep

understanding of asynchronous event loops, non-blocking I/O operations, middleware pipelines, RESTful API design conventions, structured error handling, and separation of concerns following the MVC design pattern.

- **Database Design & ORM Mastery:** Gained practical knowledge in relational schema design, third normal form (3NF) normalization, foreign key constraints, transactional consistency, and data modeling using Sequelize ORM. Learned how to optimize SQL queries, configure associations, and execute safe migrations and seeders.
- **Application Security & Authentication:** Mastered cybersecurity best practices, including implementing cryptographically signed JSON Web Tokens (JWT), password salting and hashing with bcrypt, role-based authorization filters, token expiration lifecycles, and input sanitization to protect against OWASP Top 10 vulnerabilities like SQL injection and XSS.
- **Data Visualization & Analytics:** Developed skills in transforming relational operational data into summarized analytical indicators and binding aggregated datasets to interactive Chart.js visualizations, learning how visual hierarchy and UX design principles enhance managerial decision-making.
- **System Testing & Quality Assurance:** Achieved expertise in API verification methodologies using Postman, including constructing parameter-driven test collections, scripting automated assertions, simulating edge-case payloads, inspecting response latencies, and conducting end-to-end integration testing.
- **Version Control & Collaborative Engineering:** Strengthened teamwork workflows through Git and GitHub, adopting GitFlow branching strategies, managing feature branches, conducting peer code reviews, writing standardized commit messages, and resolving complex merge conflicts across multi-developer modules.
- **Project Management & SDLC Discipline:** Experienced the full Software Development Life Cycle (SDLC) under Agile methodologies, learning how to decompose broad user requirements into manageable development sprints, adhere to design specifications, and produce formal academic and technical documentation.

CHAPTER -13 REFERENCES

Academic Textbooks and Publications

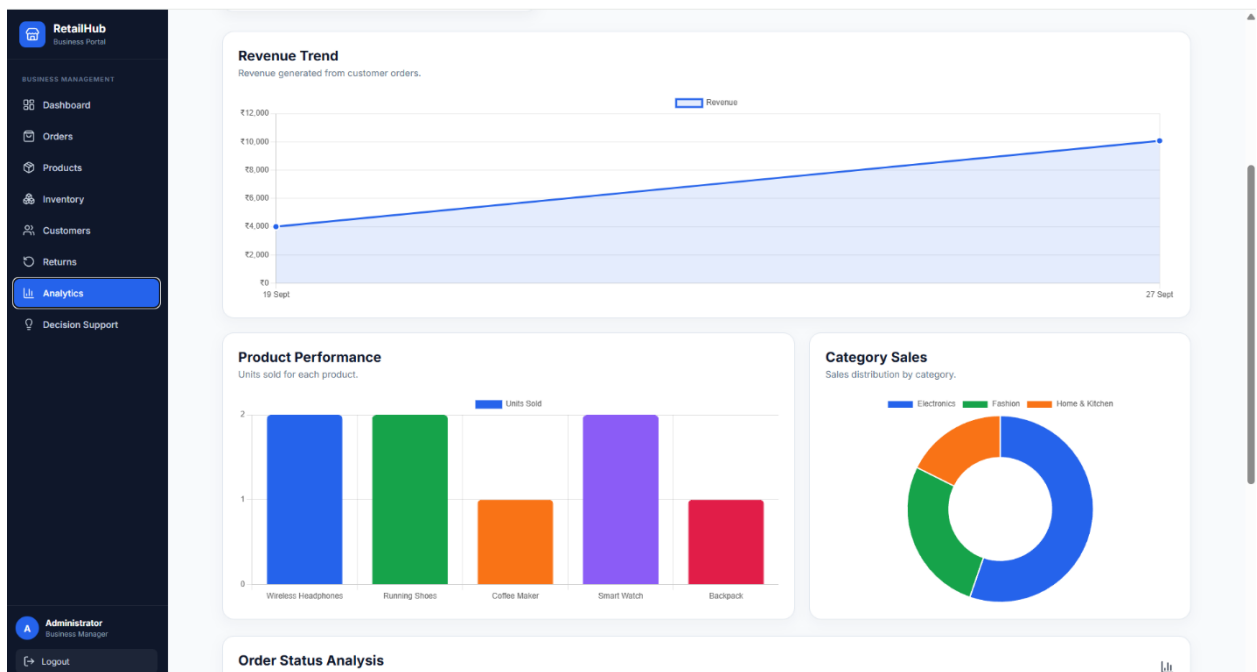
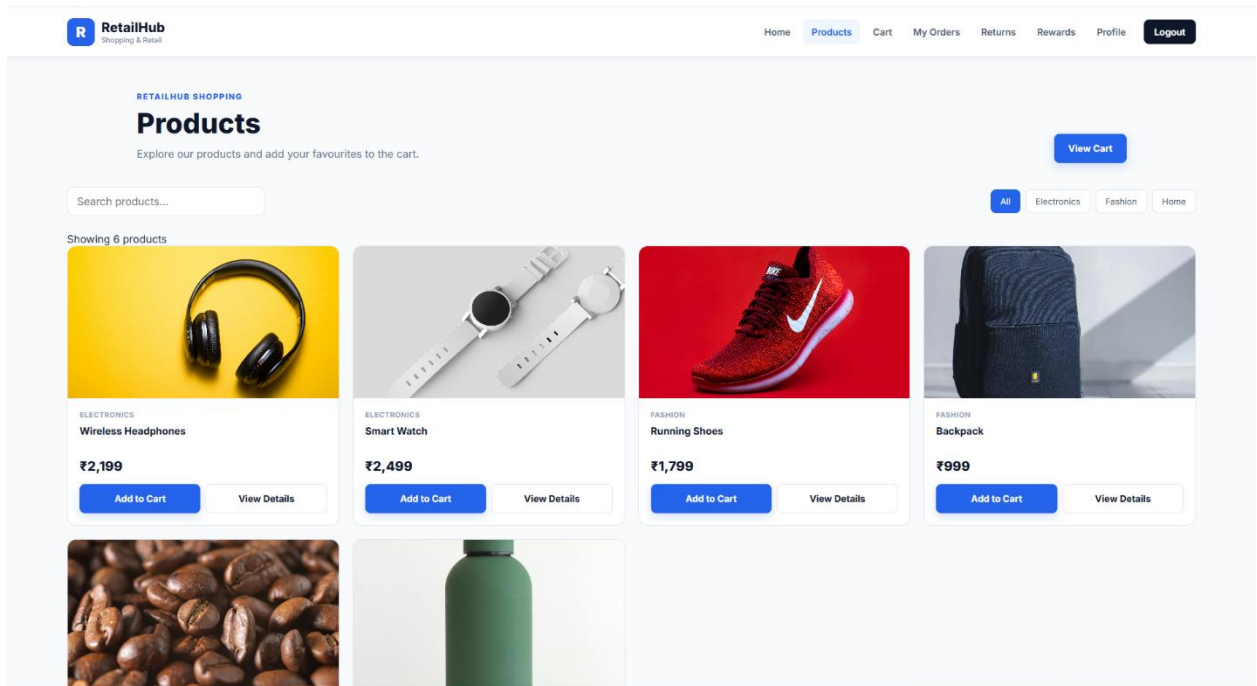
1. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
3. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
4. Banks, A., & Porcello, B. (2020). Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media.
5. Brown, E. (2019). Web Development with Node and Express: Leveraging the JavaScript Stack (2nd ed.). O'Reilly Media.
6. Silberschatz, A., Korth, H. F., & Sudarshan, P. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

Technical Documentation and Developer Frameworks

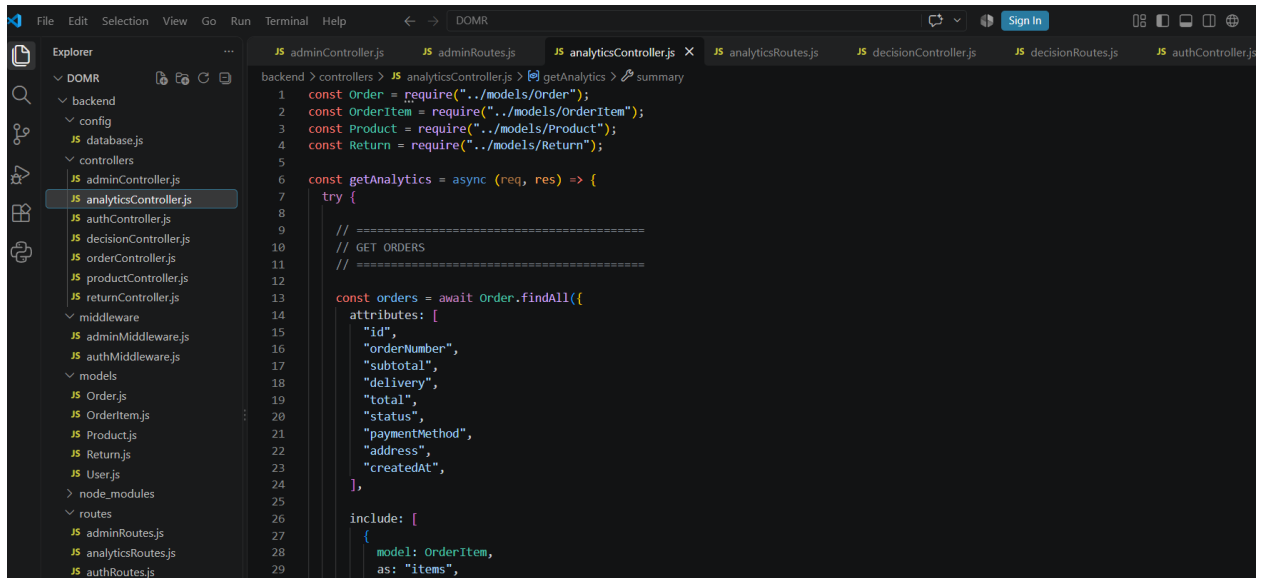
1. React.js Official Documentation. (2025). React – A JavaScript Library for Building User Interfaces. Meta Open Source. Retrieved from <https://react.dev>
2. Express.js Framework Documentation. (2025). Fast, Unopinionated, Minimalist Web Framework for Node.js. OpenJS Foundation. Retrieved from <https://expressjs.com>
3. Sequelize ORM Documentation. (2025). Promise-Based Node.js ORM for MySQL, MariaDB, PostgreSQL, and SQLite. Retrieved from <https://sequelize.org>
4. MySQL 8.0 Reference Manual. (2025). Relational Database Management System Documentation. Oracle Corporation. Retrieved from <https://dev.mysql.com/doc/>
5. Chart.js Documentation. (2025). Simple Yet Flexible JavaScript Charting for Designers and Developers. Retrieved from <https://www.chartjs.org/docs/>
6. JSON Web Token (JWT) Standards. (2025). RFC 7519: JSON Web Token Architecture and Claims Specification. Internet Engineering Task Force (IETF). Retrieved from <https://jwt.io>
7. Postman API Platform Documentation. (2025). Comprehensive API Testing, Mocking, and Environment Automation Guides. Postman Inc. Retrieved from <https://learning.postman.com>
8. Mozilla Developer Network (MDN). (2025). Web Technologies Documentation: JavaScript, HTML5, CSS3, and Fetch API Specifications. Retrieved from <https://developer.mozilla.org>
9. Node.js Documentation. (2025). Node.js v18 LTS Runtime Architecture and Asynchronous I/O APIs. Retrieved from <https://nodejs.org/docs/>
10. Axios Documentation. (2025). Promise-Based HTTP Client for the Browser and Node.js. Retrieved from <https://axios-http.com>

CHAPTER -14 APPENDICES

Screenshots of the app



Sample code snippets



The screenshot shows the Visual Studio Code editor with a project named 'DOMR'. The Explorer sidebar on the left shows the file structure, with 'analyticsController.js' selected under the 'controllers' directory. The main editor window displays the code for 'analyticsController.js'. The code defines a 'getAnalytics' function that uses 'async' and 'await' to fetch orders from a database. It includes comments for GET requests and uses 'OrderItem' as a model to include items in the response.

```
backend > controllers > JS analyticsController.js > getAnalytics > summary
1  const Order = require("../models/Order");
2  const OrderItem = require("../models/OrderItem");
3  const Product = require("../models/Product");
4  const Return = require("../models/Return");
5
6  const getAnalytics = async (req, res) => {
7    try {
8      // =====
9      // GET ORDERS
10     // =====
11
12     const orders = await Order.findAll({
13       attributes: [
14         "id",
15         "orderNumber",
16         "subtotal",
17         "delivery",
18         "total",
19         "status",
20         "paymentMethod",
21         "address",
22         "createdAt",
23       ],
24     },
25     {
26       include: [
27         {
28           model: OrderItem,
29           as: "items",
30         },
31       ],
32     });
33   } catch (error) {
34     res.status(500).json({ message: error.message });
35   }
36   res.json(orders);
37 }
```



The screenshot shows the Visual Studio Code editor with a project named 'DOMR'. The Explorer sidebar on the left shows the file structure, with 'analyticsRoutes.js' selected under the 'routes' directory. The main editor window displays the code for 'analyticsRoutes.js'. The code sets up an Express router, imports the 'getAnalytics' controller, and defines a GET route for the root path ('/') that uses 'authMiddleware' and 'adminMiddleware' before calling 'getAnalytics'.

```
backend > routes > JS analyticsRoutes.js > ...
1  const express = require("express");
2
3  const {
4    getAnalytics,
5  } = require("../controllers/analyticsController");
6
7  const authMiddleware = require("../middleware/authMiddleware");
8  const adminMiddleware = require("../middleware/adminMiddleware");
9
10 const router = express.Router();
11
12 router.get(
13   "/",
14   authMiddleware,
15   adminMiddleware,
16   getAnalytics
17 );
18
19 module.exports = router;
```

Installation/setup instructions

The Digital Order Management with Database System and Retail Analytics platform is engineered as a modern full-stack web application with a decoupled React.js frontend and a Node.js/Express.js backend connecting to MySQL Server through Sequelize ORM. The setup and local deployment can be completed systematically using the following procedure:

System Prerequisites:

Before commencing installation, ensure that Node.js (version 18.x LTS or higher), npm (version 9.x or higher), MySQL Community Server (version 8.0 or higher), and Git version control are installed and correctly configured within the system PATH environment.

Step 1: Repository Cloning and Workspace Setup

Clone the project repository to your local development environment using Git, or extract the provided project archive into a dedicated directory:

```
git clone https://github.com/retail-org/retail-order-analytics.git
cd retail-order-analytics
```

Step 2: Backend Dependency Installation and Environment Configuration

Navigate into the backend service directory and install the required server dependencies, including Express, Sequelize, mysql2, jsonwebtoken, bcryptjs, dotenv, and cors:

```
cd backend
npm install
```

Create a .env configuration file in the root of the backend folder with the necessary database and authentication credentials:

```
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=YourPassword
DB_NAME=retail_order_db
JWT_SECRET=super_secret_jwt_key_2026
```

Step 3: Database Schema Migration and Data Seeding

Ensure the local MySQL service is active, then execute the Sequelize CLI migration scripts to automatically generate all database tables, foreign key constraints, and seed initial catalog records:

```
npx sequelize-cli db:create  
npx sequelize-cli db:migrate  
npx sequelize-cli db:seed:all
```

Step 4: Launching the Backend REST API Service

Start the Node.js server in development mode. The server will initialize and begin listening for incoming REST requests on `http://localhost:5000`:

```
npm run dev
```

Step 5: Frontend Installation and Client Execution

Open a secondary terminal window, navigate into the frontend project directory, install client-side dependencies (including React, React Router, Axios, and Chart.js), and launch the development server:

```
cd ../frontend  
npm install  
npm start
```

The application will compile and automatically launch in your default web browser at <http://localhost:3000>.

User manual or guide

The Digital Order Management and Retail Analytics Portal provides a clean, responsive, and role-based user experience designed for three operational categories: Customers, Store Staff, and System Administrators.

1. Customer User Workflow:

When users access the platform, the top navigation header presents options for Login, Registration, Product Catalog, and Cart. New users can register by submitting their personal details, after which JWT authentication provides secure access to the storefront. Customers can browse available retail products, utilize category and price filters, and view real-time stock levels. Selecting an item allows adding it to the shopping cart. During checkout, users verify their shipping address and submit payment details to generate a unique Order ID. Customers can monitor their order lifecycle under the "My Orders" tab (tracking states from Pending to Processing, Shipped, and Delivered), accumulate loyalty points on successful transactions, and initiate return requests for eligible merchandise.

3. Administrator and Analytics Workflow:

Administrators authenticate to access the executive Admin Dashboard. The Analytics section renders interactive visual charts powered by Chart.js, summarizing total revenue, order volume trajectories, top-selling product breakdowns, and return percentages. Administrators can manage product listings, adjust pricing, broadcast system notifications, and consult the Decision Support System (DSS) module for automated restocking alerts and inventory turnover recommendations.

Geo Tag photos with guide



Review forms with guide signatures

